VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
DEPARTMENT OF MATHEMATICS



GRADUATION THESIS

ENSEMBLE METHOD FOR CREDIT SCORING MODEL ON HOME CREDIT DEFAULT RISK DATASET

Submitted in partial fulfillment of the requirements for the degree of
BACHELOR OF SCIENCE
IN APPLIED MATHEMATICS
SPECIALIZATION IN FINANCIAL ENGINEERING
AND RISK MANAGEMENT

Student's Name: NGUYEN THI MINH THU
Student's ID: MAMAIU19014
Thesis Supervisor: PHAM HAI HA, Ph.D

Ho Chi Minh City, Vietnam
March 2023

ENSEMBLE METHOD FOR CREDIT SCORING MODEL ON HOME CREDIT DEFAULT RISK DATASET

By

NGUYEN THI MINH THU

Submitted to DEPARTMENT OF MATHEMATICS,
INTERNATIONAL UNIVERSITY, HO CHI MINH CITY
in partial fulfillment of requirements for the degree of
BACHELOR OF SCIENCE
IN APPLIED MATHEMATICS
SPECIALIZATION IN FINANCIAL ENGINEERING
AND RISK MANAGEMENT
July 2023

Signature of Student: _____

NGUYEN THI MINH THU

Certified by: _____

PHAM HAI HA, PhD.
Thesis Supervisor

Approved by: _____

Prof. Pham Huu Anh Ngoc
Head of Department of Mathematics

ENSEMBLE METHOD FOR CREDIT SCORING MODEL ON HOME CREDIT DEFAULT RISK DATASET

By

NGUYEN THI MINH THU

Submitted to DEPARTMENT OF MATHEMATICS,
INTERNATIONAL UNIVERSITY, HO CHI MINH CITY
in partial fulfillment of requirements for the degree of
BACHELOR OF SCIENCE
IN APPLIED MATHEMATICS
SPECIALIZATION IN FINANCIAL ENGINEERING
AND RISK MANAGEMENT

ABSTRACT

Access to credit is a vital consideration for many individuals, with credit scores playing a critical role in determining loan eligibility. In Vietnam, credit history data is stored in two institutions, the Private Credit Bureau (PCB) and the Credit Information Center (CIC). These institutions record credit data and provide comprehensive information to financial institutions, which helps assess loan applications and predict a customer's ability to repay loans or probability of default.

This thesis investigates the effectiveness of machine learning approaches for building a predictive model to estimate the likelihood of customer default. An ensemble of machine learning techniques, including traditional, deep, dynamic, and boosting algorithms, is employed to forecast the default status and repayment ability of a loan. The ensemble models outperform conventional models, achieving AUC scores of approximately 78% versus 50%.

The research utilized several techniques, including cross-validation and resampling methods. The effectiveness of these methods is evaluated using the metric AUC. The findings of this study can help lenders improve the accuracy of their credit scoring models while increasing the loan approval rate.

Keywords: Credit Scoring Model, Probability of Default, Ensemble Methods, Stacking, Bagging, Boosting, Traditional Machine Learning Model.

ACKNOWLEDGEMENTS

Firstly, I am deeply grateful to Ms. Pham Hai Ha, Ph.D, my supervisor, for her guidance, support, and encouragement. Her expertise in the field of machine learning and insightful feedback were invaluable in bringing this thesis to completion. I am thankful for the time and effort she invested in my research and for inspiring me to pursue a career in machine learning through her mentorship.

Secondly, I am extremely grateful to all the teachers and professors in the Mathematics Department for their dedication to my education and deepening my understanding of mathematics. Their enthusiasm and expertise have played a significant role in shaping my academic journey.

Thirdly, I am extremely grateful to Mr. Phuc, my team leader, and Mr. Hoang, my manager at M Service for providing me with a valuable opportunity to gain experience in the consumer lending industry. Furthermore, I truly appreciate the support from my colleagues. Their guidance and support, which not only helped me to develop my analytical skills but also inspired me to pursue a career in data analysis.

Fourthly, I would like to extend my heartfelt appreciation to my family and friends for their unwavering support and encouragement throughout my academic journey. Their love and motivation have sustained me during the challenging times, and their belief in my abilities has inspired me to strive for excellence. I am particularly grateful to my parents, who have always been my biggest supporters and have sacrificed so much to ensure that I have had every opportunity to succeed.

Last but not least, I would like to extend my appreciation to Kaggle, Home Credit, and International University for providing resources and opportunities that enabled me to pursue my academic goals. Their generosity has been instrumental

in making this research possible, and I am grateful for the invaluable resources they offered.

In conclusion, I am grateful to all those who have supported me throughout this journey. This thesis represents a culmination of my academic efforts, and I am honored to have had the opportunity to complete this research.

ABBREVIATIONS

AUC	Area Under the Receiver Operating Characteristics Curve
***k*NN**	k-Nearest Neighbors
SVM	Support Vector Machine
AdaBoost	Adaptive Boosting
GBoost	Gradient Boosting
XGBoost	Extreme Gradient Boosting
Light GBM	Light Gradient Boosting Machine
RUS	Random Under Sampling
ROS	Random Over Sampling
SMOTE	Synthetic Minority Over-Sampling Technique

Contents

Abstract **1**

1 Introduction **10**

2 Literature review **16**

3 Research Methodology **23**

3.1 Baseline models 24

3.1.1 Logistic Regression 24

3.1.2 k-Nearest Neighbors (kNN) 26

3.1.3 Support Vector Machine (SVM) 29

3.1.4 Decision Tree 33

3.1.5 Neural Network 37

3.2 Stacking Algorithms 40

3.3 Bagging-based Ensemble Algorithms 41

3.3.1 Random Forest 42

3.3.2 Bagged Neural Network 43

3.4 Boosting Algorithms 44

3.4.1 Adaptive Boosting (AdaBoost) 45

3.4.2 Gradient Boosting (GBoost) 48

3.4.3 Extreme Gradient Boosting (XGBoost) 50

3.4.4 Light Gradient Boosting Machine (LightGBM) 54

3.4.5 Summary 58

3.5 Evaluation metrics 59

4 Empirical Results **61**

4.1 Explore data . . . 61

4.1.1 Data structure and description . . . 62

4.1.2 Descriptive Statistics . . . 64

4.2 Pre-processing data . . . 68

4.2.1 Feature engineering . . . 68

4.2.2 Slicing data technique . . . 69

4.2.3 Resampling Methods . . . 69

4.3 Experimental Setup . . . 70

4.3.1 Cross-validation . . . 70

4.3.2 Baseline models . . . 72

4.3.3 Ensemble models . . . 74

4.4 Results . . . 76

4.5 Limitations . . . 78

5 Conclusion and Discussion **81**

List of Figures

3.1 A simple decision tree. 34
3.2 The visualization of a single-layer neural network. 38
3.3 AUC-ROC Curve. 60

4.1 Data Structure Diagram. 62
4.2 The distribution of defaults and non-defaults in the training set. . . . 65
4.3 The distribution of defaults and non-defaults in the training set with respect to contract types. 66
4.4 The distribution of defaults and non-defaults in the training set is broken down with respect to genders. 67
4.5 The distribution of defaults and non-defaults in the training set with respect to family status. 67
4.6 The histogram of the distribution of loan amount based on the target variable. 68
4.7 K Fold Visualization Cross-Validation. 71
4.8 Stratified K Fold Visualization Cross-Validation. 72

List of Tables

4.1 The proportion of missing values on each variable in training and testing datasets. 64
4.2 The performance of ensemble methods on testing dataset with feature engineering. 77

Chapter 1

Introduction

Background and Context

Over the past few years, the availability of consumer loans has played a significant role in enhancing financial inclusion and providing access to financial services for low- and middle-income individuals. This has also helped prevent vulnerable individuals from falling into the trap of resorting to "black credit" with exorbitant interest rates. As the demand for consumer loans has increased, numerous consumer finance companies in Vietnam have emerged, offering competitive fees and interest rates to entice customers. This has led to intense competition among these companies, putting pressure on them to generate revenue by attracting creditworthy customers. However, aggressive marketing can also result in attracting bad customers, which creates challenges for risk managers who must not only determine which attributes indicate a good credit candidate but also minimize the cost of evaluating each client. To address this issue, credit scoring systems offer a reliable and empirical solution for businesses to accurately segment customers and offer suitable loan packages.

Credit scoring is a mechanism for evaluating the creditworthiness of a borrower or institution based on statistical analysis conducted by lenders or financial institutions. This scoring system uses credit scorecards, which provide lenders with an overview of the borrower's creditworthiness, enabling them to make informed decisions about the appropriate interest rate, loan amount, and loan duration. Typically, these scorecards rely on statistical models that consider a variety of factors

about the borrower, including their income, employment status, credit history, and other relevant information. The scorecard assesses the probability of default for the borrower by analyzing this information, which is then used to determine their credit score.

During the 1800s, credit bureaus started to emerge. Credit reporting is invented as part of America's transition to capitalist modernity. However, its past has been both frightening and empowering, allowing millions to realize the American Dream through credit. Additionally significant is that it has left the majority of Americans with a permanent "financial identity" that is a mark that represents bad behavior in the past and encourages good behavior in the future (Kiviat 2019).

In America in the 18th century, country storekeepers obtained loans by requesting respectable neighbors to attest for them to lenders and merchants or urban creditors scoured remote country friends for hearsay and tales about credit applicants. However, credit reporting started to modernize in the 1820s as the volume of commercial deals rendered the outdated method unwieldy. Loans became a riskier prospect due to new bankruptcy rules. A number of experiments were conducted to standardize credit assessment as a consequence. Even though the focus of these trials was on industrial credit loans to businesses, they would have a significant impact on the development of consumer credit ratings in the future. In 1857, a man named Lewis Tappan founded the Mercantile Agency, which later became known as Dun & Bradstreet. Tappan's company created a system to evaluate the creditworthiness of businesses based on their financial stability, reputation, and other factors. Tappan's organization gathered information from reporters across the nation and organized it in sizable ledgers in New York City. These reports' subjectivity had two significant effects. Firstly, it reinforced the already existing social hierarchy, serving as an early form of discriminatory lending practices based on geography. Secondly, the mixture of unverified information present in the initial reports was challenging to convert into practical insights. In the late 19th century, retailers began using credit to sell goods to customers. However, as the number of credit transactions grew, the need for a more systematic way of evaluating creditworthiness became apparent. This led to the development of personal credit reports, which were created by collecting

information from a variety of sources, including banks, merchants, and employers. Commercial credit rating, pioneered by Bradstreet in 1857 and formalized by R.G. Dun and Company in 1864, converted borrowers' information into actionable financial "facts" using a pseudo-scientific approach that lasted until the twentieth century (Fitzpatrick 2015).

In the early 20th century, a number of credit bureaus were established to gather and distribute credit information. One of the most significant of these was the Retail Credit Company, which later became known as Equifax. Retail Credit Company was the first to use a scoring system to evaluate creditworthiness. The system, which was called the BEACON system, used a set of mathematical formulas to generate a score based on the borrower's credit history. In the 1950s, Bill Fair, an engineer, and Earl Isaac, a mathematician, collaborated together to establish the Fair Isaac Corporation's credit scoring system, which is also known as FICO score. The objective of this system was to create a standardized and objective credit scoring system. Nowadays, FICO score is the most commonly used credit scoring system in the financial industry especially in the United States. VantageScore was launched in 2006 is another well-known credit scoring model that is extensively utilized. This model was developed by the three major credit reporting agencies, namely TransUnion, Experian, and Equifax. The formulas of these scorecard models can vary greatly. The process of experimenting with segmentation, determining the optimal number and configuration has contributed to the development of credit scorecards. The analysis carried out can uncover valuable insights into the risk behavior of customers, which can lead to a better understanding of their needs and improved strategy development.

Problem Statement

The primary objective of this thesis is to evaluate the effectiveness of various machine learning techniques, particularly ensemble techniques in constructing a robust model that can accurately predict the likelihood of customer default on the Home Credit dataset. The motivation for undertaking this research is that although ensemble algorithms have become increasingly popular, especially in credit scoring for

tech companies in the lending industry, they are limited by Vietnamese law when it comes to accessing confidential data such as customer credit history, income, and social insurance information required for creating effective credit scorecards. Conversely, traditional banks in Vietnam have the system storing citizens' credit histories, verified payslips, and social insurance information. However, these banks often rely on traditional credit scoring models, such as logistic regression, which may not be appropriate for handling the complexity of large and intricate credit data in today's technological age. Furthermore, the performance of traditional models may be affected by imbalanced data, noisy data, or non-linear relationships between variables. Ensemble methods have shown promise in addressing these challenges and improving the predictive accuracy of credit scoring models. Despite the popularity of ensemble methods in credit scoring, especially for big and complex data, there is a lack of consensus on which methods are most effective and how they should be implemented in practice. This thesis seeks to evaluate the effectiveness of various ensemble methods in credit scoring and to provide guidelines for practitioners to select and apply these methods in an efficient and accurate manner.

Research Question

The objective of this study is to investigate the effectiveness of ensemble methods in credit scoring, which has received limited attention in the context of big data. This will be achieved by addressing the following research questions.

- Which ensemble techniques are the most efficient in handling the difficulties presented by imbalanced data, noisy data, and non-linear relationships between variables?
- How does the AUC score of ensemble methods compare to standalone models?
- What are the best practices for implementing ensemble methods, taking into account factors such as AUC score, interpretability, and computational efficiency, that practitioners should be aware of?
- What is the impact of different resampling and cross-validation techniques on the performance and interpretability of ensemble methods?

- How can the results of this study inform best practices in credit scoring?

Objectives and Scopes

The primary aim of this research is to assess the efficacy of various ensemble methods in credit scoring, and to provide practical recommendations to practitioners on selecting and utilizing these methods in a precise and efficient manner. To accomplish this objective, the study will compare the AUC score of ensemble methods to traditional machine learning models using the Home Credit Default Risk dataset, which was featured in a Kaggle competition in 2018. This dataset is extensive and contains various alternative data sources, including transactional and telco data, that are used to predict the repayment capacity of clients and ensure a positive loan experience for this customer group.

Furthermore, the study will examine which ensemble methods are most successful in dealing with the challenges posed by imbalanced data, noisy data, and non-linear relationships between variables in credit scoring. Additionally, the study will explore how the performance and interpretability of ensemble methods vary with different re-sampling and cross-validation techniques. The investigation will be confined to the Home Credit Default Risk dataset, focusing specifically on the use of ensemble methods in forecasting the risk of default for consumer loans issued by Home Credit. This targeted approach aims to provide a more concentrated and thorough analysis of the research topic.

The outcomes of this research will be beneficial to financial institutions and practitioners in the lending industry, especially in the era of big data, and will contribute to the development of best practices for credit scoring models.

Significance of the study

The importance of this research lies in its ability to aid in the development of best practices for credit scoring models, especially in the domain of ensemble methods. Although ensemble methods have gained popularity in credit scoring, their effectiveness in this field remains largely unexplored. This study aims to fill this gap by comparing the effectiveness of different ensemble techniques with traditional

models through the use of various re-sampling and cross-validation techniques. As a result, the study will offer useful insights into the practical implementation of ensemble methods in credit scoring, helping practitioners choose and apply these methods accurately and efficiently. The results of this study will be particularly relevant in the age of big data, where the employment of alternative data sources and machine learning techniques is increasingly important in credit risk evaluation. Ultimately, the outcomes of this study will aid in the creation of more precise, effective, and transparent credit scoring models, which can improve access to credit for individuals and businesses, promote financial inclusion, and stimulate economic growth.

Structure of thesis

The following chapters of this thesis are organized as follows. Chapter 2 provides a review of literature related to applying ensemble methods in credit scoring. Chapter 3 outlines the research methodology, which includes bagging, boosting, and stacking algorithms, as well as standalone machine learning algorithms. Chapter 4 presents the empirical results, which include details on data exploring, data preprocessing, model implementation, evaluation, and limitations. Finally, Chapter 5 discusses the implications of the findings and suggests future research directions related to this topic.

Chapter 2

Literature review

Ensemble methods: Foundations and algorithms by Zhou (2012) discussed the history of ensemble methods. According to the authors, the use of multiple models had been common in human society for a while, but it was difficult to establish a precise beginning for the history of ensemble methods. However, it is clear that the popularity of ensemble methods as a research topic has surged since the 1990s. Machine learning, pattern recognition, data mining, neural network, and statistics have shown a keen interest in ensemble methods. Researchers have investigated ensemble methods from a range of perspectives. The principle of multiple explanations, introduced by the Greek philosopher Epicurus (341 - 270 B.C.), predates the widely known Occam's razor principle in scientific research. Occam's razor favors simpler hypotheses over complex ones that could explain empirical observations equally well. Asmis (1984) explained that the principle of multiple explanations advocates retaining all hypotheses that are consistent with empirical observations.

Ensemble methods have become a prominent learning paradigm since the 1990s, largely due to the significant contributions of two pioneering works. Firstly, Breiman (1996) discovered the empirical approach that combining a set of classifiers can lead to predictions that are generally more accurate than those produced by the best-performing single classifier. Secondly, according to Freund, R. Schapire, and Abe (1999), the theoretical approach to ensemble methods was revealed. The approach showed that weak learners can be boosted to become strong learners, which are highly desired but difficult to obtain. On the other hand, weak learners are more

accessible in practical situations. This discovery provides a promising solution for producing strong learners using ensemble methods. Over time, a variety of ensemble methods have been created and utilized in credit scoring, such as Random Forest, Stacking, and AdaBoost. It has been demonstrated that utilizing these techniques can increase the precision and strength of credit scoring models, which is crucial in the financial industry, particularly for credit scoring issues. Nevertheless, employing ensemble methods to tackle these problems may encounter challenges, especially when the training data is imbalanced in terms of class distribution. The challenge of handling outliers and imbalanced data is critical in credit data analysis, making an ensemble model for credit scoring a promising approach to address these issues and improve the model's adaptability to outliers and imbalanced data.

Imbalanced data is a common issue in credit scoring, where there may be a disproportionate number of good or bad credit instances, which can lead to biased models and inaccurate predictions. There exists various methods to address the issue of class imbalance before training models.

Various resampling techniques, including Random Under Sampling (RUS), Random Over Sampling (ROS), and the Synthetic Minority Over-Sampling technique (SMOTE), were developed to address imbalanced data. He and Garcia (2009) presented a comprehensive study of learning from imbalanced data, where the number of instances in one class was significantly smaller than the other. "Benchmarking State-of-the-Art Classification Algorithms for Credit Scoring" by Baesens et al. (2003) reviewed the performance of different classification algorithms for credit scoring based on a benchmarking study. The study provided a thorough assessment of imbalanced learning techniques for credit scoring. The authors compared various methods such as resampling, cost-sensitive learning, and ensemble approaches and analyzed how different evaluation measures affect their performance. Additionally, the study revealed that SMOTE, when compared to other resampling techniques like Random Over-Sampling and Random Under-Sampling, could enhance the performance of classification algorithms in credit scoring datasets. Consequently, the research offered significant insights into the efficacy of imbalanced learning techniques for credit scoring, and the authors suggested that these techniques could help

improve the performance of credit scoring models in highly imbalanced datasets and prevent models from being biased towards the majority class. The study emphasized the importance of selecting appropriate evaluation measures, such as F1-score, when evaluating the performance of classification algorithms in imbalanced datasets.

Cross-validation is essential in class imbalanced data to evaluate model performance fairly and accurately, avoiding biases towards the majority class. Santos et al. (2018) explored the issue of overoptimistic and overfitting approaches in cross-validation for imbalanced datasets. The research introduced a new method called "stratified cross-validation" to tackle the issue of imbalanced datasets. This approach aimed to maintain the distribution of the minority class in every fold of the cross-validation process. The study examined the performance of this method on several imbalanced datasets using different classification algorithms. The findings indicated that the stratified cross-validation method outperformed other cross-validation approaches in terms of accuracy, AUC, and sensitivity. Additionally, the study showed that this method could prevent the common problems of overoptimistic and overfitting results in imbalanced datasets. In general, the research explained the importance of utilizing appropriate cross-validation techniques in imbalanced datasets to avoid overoptimistic and overfitting models. The implications of these findings are significant for machine learning researchers and practitioners dealing with imbalanced datasets, particularly in domains like credit scoring, fraud detection, and disease diagnosis.

Ensemble techniques have been proposed for credit scoring in response to the limited accuracy of conventional models, such as Logistic Regression, Naive Bayes, and SVM, in assessing creditworthiness. The poor performance of these models has been extensively studied in several papers.

Khandani, Kim, and Lo (2010) argued that traditional credit-scoring models, such as logistic regression, suffer from a number of limitations. They noted that these models rely on subjective human judgment in selecting the variables to include in the model, and that they were often unable to accurately capture complex relationships between variables. Furthermore, these models might not perform well when there was a significant change in the economic environment.

Yuelin Wang et al. (2020) compared the performance of different machine learning models for credit risk assessment using bank loan data. To achieve this goal, the study employed various machine learning models, including logistic regression, decision tree, random forest, gradient boosting, and neural network. The study evaluated the performance of these models based on different performance metrics such as accuracy, AUC, and F1-score. They found that Naive Bayes performed well when there was a large amount of data, but it was outperformed by logistic regression and decision tree algorithms when the data was limited. The authors also noted that Naive Bayes assumes independence between the input variables, which might not always be true in practice. They concluded that Naive Bayes might not always be the best choice for credit scoring and that other algorithms should be considered based on the specific characteristics of the data.

"A comparative study of credit scoring methods based on support vector machines" by Yan Liu et al. (2009) compared the performance of several data mining techniques, including SVM, in predicting credit risk for a Chinese bank. The results of the study showed that while SVM performed well in terms of accuracy, they were outperformed by other techniques such as decision tree and neural network when it came to classification performance measures such as precision and recall. Additionally, the authors noted that SVM could be sensitive to the choice of kernel function and the selection of hyperparameters, which could affect their performance.

In contrast to the performance of conventional models, recent advances in machine learning have resulted in improvements, particularly with ensemble methods consisting of bagging, boosting, and stacking. A variety of studies have shown that ensemble credit scoring models outperform individual models in a significant way, as evidenced by studies.

"Benchmarking State-of-the-Art Classification Algorithms for Credit Scoring" by Baesens et al. (2003) reviewed the performance of different classification algorithms for credit scoring based on a benchmarking study. The paper utilized data from a Belgian bank, which included both demographic and credit-related information, to compare the performance of nine different algorithms (Logistic Regression, Decision Tree, Neural Network, k-Nearest Neighbors, Support Vector Machine, Random For-

est, Gradient Boosting, Bayesian Network, and rule-based classifiers). The study found that the gradient boosting algorithm outperformed the other algorithms in terms of accuracy and AUC. Additionally, the study explored that feature selection techniques, such as recursive feature elimination, could improve the performance of the classification algorithms. In summary, the study highlighted the importance of selecting appropriate classification algorithms for credit scoring applications.

Dumitrescu et al. (2022) introduced a new method for credit scoring penalised logistic tree regression by combining logistic regression with a non-linear decision tree approach. The study proposed a novel approach that combined logistic regression with decision tree effects, referred to as the "hybrid model". The decision trees affected are incorporated into the logistic regression model as non-linear transformations of the input variables. The outcomes demonstrated that the hybrid model outperformed both the logistic regression model and a traditional decision tree model in terms of accuracy and AUC. Moreover, the hybrid model was shown to be more effective in capturing non-linear relationships between the input variables and the credit risk. In conclusion, the research highlighted the potential of machine learning techniques such as decision tree effects to improve the performance of credit scoring models.

A data-driven ensemble classifier for credit scoring analysis was proposed by Hsieh and Hung (2010). The proposed ensemble classifier combined the strengths of multiple classification techniques to achieve higher accuracy in credit scoring compared to individual classifiers. The authors utilized four classifiers, namely Logistic Regression, Decision Tree, k-Nearest Neighbors, and Support Vector Machine, to build the ensemble model. They used a dataset of over 30,000 observations from a major bank in Taiwan to train and test the ensemble model. The results showed that the proposed ensemble model outperformed individual classifiers and achieved higher accuracy in credit scoring. The authors also provided a sensitivity analysis of the ensemble model to identify the most influential input variables.

W. Liu, Fan, and Xia (2022) suggested a new ensemble approach for credit scoring that utilized a tree-based heterogeneous cascade ensemble model. The model combined the strengths of multiple base classifiers, including Decision Trees, Ran-

dom Forest, Support Vector Machine, and k-Nearest Neighbors, to improve credit scoring accuracy. The approach was designed to handle imbalanced data and to reduce the number of false positives and false negatives. The authors conducted experiments using several real-world credit datasets and compared the performance of their approach to several other ensemble methods. The results showed that the suggested model outperformed the other methods in terms of accuracy, precision, recall, and AUC. The research demonstrated the potential of using a heterogeneous cascade ensemble approach for credit scoring systems.

Yadong Wang et al. (2022) proposed a deep reinforcement learning approach with a confusion matrix-based dynamic reward function for customer credit scoring. The proposed model aimed to improve the credit scoring accuracy by dynamically adjusting the confusion matrix-based reward function, which considered both false positives and false negatives. The model was evaluated on a real-world dataset, and the results demonstrated that it outperformed other traditional machine learning models, such as Logistic Regression, Decision Tree, and Random Forest. The study concluded that the proposed model could effectively capture the complex relationships between the input features and the creditworthiness of customers, leading to more accurate credit scoring.

Abellán and Mantas (2014) presented a study that aimed to enhance the performance of ensemble classifiers for bankruptcy prediction and credit scoring. To achieve this objective, several techniques were utilized, including data preprocessing, dimensionality reduction, and ensemble learning. Notably, the study introduced a new methodology referred to as the "hybrid ensemble" that integrated different types of classifiers to optimize performance. The results of the paper indicated that the hybrid ensemble approach surpassed individual classifiers and conventional ensemble methods in terms of accuracy and AUC. Furthermore, the study revealed that data preprocessing techniques, such as oversampling and undersampling, could enhance classifier performance on datasets that exhibited class imbalance. Overall, this study underscored the significance of ensemble methods in bankruptcy prediction and credit scoring and demonstrated the efficacy of the hybrid ensemble approach in improving model performance.

Overall, these studies suggest that ensemble methods can improve the performance of credit scoring models in predicting credit risk. By combining multiple models, ensemble methods can take advantage of the strengths of each individual model and produce a more robust and reliable credit scoring model.

Chapter 3

Research Methodology

Ensemble modeling is a technique that combines the predictions of multiple machine learning models to produce a more accurate and robust model. This approach has gained popularity in the field of credit scoring, where the main objective is to predict whether a borrower will default on a loan or not. The key idea behind ensemble modeling for credit scoring is to develop several base models using different algorithms or subsets of the available data, and then combine their predictions using a weighted average or a voting system. The factors that influence the effectiveness of ensemble learning include the diversity of the base learners, the independence of the base learners, the size of the ensemble, the aggregation method, and the quality and quantity of training data. Furthermore, the ensemble scoring approaches can be divided in to two kinds by W. Liu, Fan, and Xia (2022) which are homogeneous, i.e., using the same feature selection method with different training data and distributing the dataset over several nodes; and heterogeneous, i.e., using different feature selection methods with the same training data. The rationale behind combining rankings of features that contain all the ordered features approach is that different models might capture different patterns in the data, and by combining them, we can reduce the variance and increase the accuracy of the overall model. Specifically, the weak learners can either have a high bias or high variance, and there are trade-offs between bias and variance. Thus, the ensemble method objective is to derive a model that will improve the performance in three ways by:

- Reducing the variance of weak learners.

- Reducing the bias of weak learners.
- Improving the overall accuracy of strong learners.

These ensemble methods are classified into three groups based on their integration strategy: bagging, boosting, and stacking. Before exploring them, it is beneficial to initially investigate several common classification algorithms that can act as weak learners in ensemble algorithms. Within each ensemble method section, there are examples demonstrating how each ensemble method functions with these weak learners.

3.1 Baseline models

Machine learning is a field that explores the development and analysis of mathematical models and algorithms to uncover insights from data. Two main approaches in machine learning are supervised and unsupervised learning, which are defined by the presence of labeled data. In supervised learning, models are trained on labeled data where the target outcomes are already known. In contrast, unsupervised learning involves training models on unlabeled data to discover patterns, structures, or relationships in the data. Common models for supervised learning include regression and classification, while clustering and association are examples of unsupervised learning. This thesis mainly focuses on classification problems with labeled data, and therefore a supervised machine learning approach will be considered. The thesis will explore five commonly-used supervised models, namely Logistic Regression, k-Nearest Neighbors, Support Vector Machine, Decision Tree, and Neural Network models, for solving classification problems.

3.1.1 Logistic Regression

Logistic Regression is a statistical method used for binary classification, where the predicted outcomes are derived from one or multiple predictor variables (Hastie et al. 2009). The model explains the relationship of the predicted outcomes given the input variables.

Consider the binary outcomes is either 0 or 1, and the input matrix of size $(p+1) \times m$ where p is the number of features and m is the number of observations, the logistic regression model of class 1 with information contained in vector x_i, where $i = \overline{1, m}$ has the form follows:

$$P(y_i = 1|x_i) = \frac{e^{\beta_0 + \beta_1^T x_i}}{1 + e^{\beta_0 + \beta_1^T x_i}},$$

which can be rewritten as

$$\ln \left[\frac{P(y_i = 1|x_i)}{1 - P(y_i = 1|x_i)} \right] = \beta_0 + \beta_1^T x_i$$

where β_0, and β_1^T are the estimated coefficients and β_1 is the $(p+1) \times 2$ matrix since there are two outcome values.

Let $\beta = \{\beta_0, \beta_1\}$ which contains $(p + 1)$ nonlinear equations, and the vector of inputs x_i is assumed to include the constant term 1 to accommodate the intercept. The log-likelihood function can be rewritten as

$$\begin{aligned} l(\beta) &= \sum_{i=1}^{m} y_i(\beta_0 + \beta_1^T x_i) - \sum_{i=1}^{m} \log(1 + e^{\beta_0 + \beta_1^T x_i}) \\ &= \sum_{i=1}^{m} \left[y_i \beta^T x_i - \log(1 + e^{\beta^T x_i}) \right]. \end{aligned}$$

To maximize the log-likelihood, the partial derivative with respect to β is taken:

$$\frac{\partial l(\beta)}{\partial \beta} = \sum_{i=1}^{m} x_i \left(y_i - \frac{e^{\beta^T x_i}}{1 + e^{\beta^T x_i}} \right) = \sum_{i=1}^{m} x_i [y_i - P(y_i = 1|x_i)] = 0. \tag{3.1}$$

The equation (3.1) is solved by using Newton–Raphson by using second derivative or Hessian matrix:

$$\frac{\partial^2 l(\beta)}{\partial \beta \partial \beta^T} = -\sum_{i=1}^{m} x_i x_i^T P(y_i = 1|x_i)[1 - P(y_i = 1|x_i)]. \tag{3.2}$$

By the Newton–Raphson method, the β_{new} can be estimated by the β_{old} as follows

$$\beta_{new} = \beta_{old} - \left(\frac{\partial^2 l(\beta)}{\partial\beta\partial\beta^T}\right)^{-1}\left(\frac{\partial l(\beta)}{\partial\beta}\right). \tag{3.3}$$

Let $\boldsymbol{y}$ be the vector of y_i values, $\boldsymbol{p}$ be the probability $P(y_i = 1|x_i)$ vector with ith element, $\boldsymbol{X}$ be the input matrix of size $m \times (p+1)$ of x_i values, $\boldsymbol{W}$ be the $m \times m$ diagonal matrix of weights with ith diagonal, then (3.1), and (3.2) can be rewritten as:

$$\frac{\partial l(\beta)}{\partial\beta} = \boldsymbol{X^T}(\boldsymbol{y} - \boldsymbol{p});\ \frac{\partial^2 l(\beta)}{\partial\beta\partial\beta^T} = -\boldsymbol{X^T W X}. \tag{3.4}$$

Substituting (3.4) into (3.3), then

$$\begin{aligned}\beta_{new} &= \beta_{old} + (\boldsymbol{X^T W X})^{-1}\boldsymbol{X^T}(\boldsymbol{y} - \boldsymbol{p}) \\ &= (\boldsymbol{X^T W X})^{-1}(\boldsymbol{X^T W X})\beta_{old} + (\boldsymbol{X^T W X})^{-1}\boldsymbol{X^T W W^{-1}}(\boldsymbol{y} - \boldsymbol{p}) \\ &= (\boldsymbol{X^T W X})^{-1}(\boldsymbol{X^T W})[\boldsymbol{X}\beta_{old} + \boldsymbol{W}^{-1}(\boldsymbol{y} - \boldsymbol{p})].\end{aligned}$$

3.1.2 *k*-Nearest Neighbors (*k*NN)

In the k-nearest neighbors prediction function, a predictor vector is used to predict a target variable. Specifically, the target variable is categorical, representing a label that identifies the group to which the observation belongs. By using the predictor vector and the k-nearest neighbors prediction function, a prediction of membership can be generated. It can also be used to predict quantitative target variables. Despite its simple concept and computational approach, it is known to achieve comparable accuracy with more complex prediction functions. The k-nearest neighbors prediction function relies on a collection of training observations comprising target and predictor vector pairs. To make a prediction for a new target, the function identifies the training observations closest to the target, based on their predictor values. If the target is a categorical variable representing group membership, the function predicts that the target belongs to the most frequently occurring group among the nearest neighbors. For a quantitative target, the function estimates the value by taking the average of the target values associated with the nearest neighbors

(Steele, Chandler, and Reddy 2016).

This thesis primarily concerns the classification problem, it is more suitable to discuss how kNN deals with categorical labels.

Input:

- Data set $D = \{\boldsymbol{x_i}, y_i\}_{i=1}^{m}$

  where

  $\boldsymbol{x_i}$ has n features;

  m be the number observations;

  $y_i \in \{0, 1\}$ be the binary outcome.

- The k observations in the training set that are most similar to the test observation $(\boldsymbol{x_0}, y_0)$ based on the distances between $\boldsymbol{x_0}$ and $\boldsymbol{x_1}, \ldots, \boldsymbol{x_m}$.

Process:

- There are multiple metrics to compute the distance between $\boldsymbol{x}_i$ and $\boldsymbol{x_0}$ including Euclidean distance, Manhattan distance, Hamming distance, etc. The most common metric used for this purpose is the Euclidean distance, which is computed as follows:

$$dist_E(\boldsymbol{x_i}, \boldsymbol{x_0}) = \left[\sum_{j=1}^{n}(x_{i,j} - x_{0,j})^2\right]^{1/2}.$$

- Scaling the distance metrics since with the squared difference, the variable with the largest value of variance will have an enormous impact on determining the difference between $\boldsymbol{x}_i$ and $\boldsymbol{x_0}$ which is independent with the variable's informative value in terms of prediction. The scaling function is

$$dist_E(\boldsymbol{x_i}, \boldsymbol{x_0}) = \left[\sum_{j=1}^{n}\left(\frac{x_{i,j} - x_{0,j}}{s_j}\right)^2\right]^{1/2}$$

  where s_j is the estimated standard deviation of the jth predictor variable.

- Sorting the value of $dist_E(\boldsymbol{x_i}, \boldsymbol{x_0})$ ascending.

- Choosing top k-nearest observations which have the smallest distance value.
- Consider an indicator function as follows:

$$I_j(y) = \begin{cases} 1 & \text{if } y = j \\ 0 & \text{if } y \neq j \end{cases}$$

where $j = \{0, 1\}$.

The fraction of group j members among the k nearest neighbors is defined as the estimated likelihood of membership in group j:

$$\hat{Pr}(y_0 = j \mid \boldsymbol{x_0}) = \frac{q_j}{k} = \frac{1}{k} \sum_{i=1}^{k} I_j(y_{[i]})$$

where q_j is the number of the k nearest neighbors that belongs to classifier j.

The estimated probabilities can be written as

$$\hat{p_0} = \{\hat{Pr}(y_0 = 0 \mid \boldsymbol{x_0}), \hat{Pr}(y_0 = 1 \mid \boldsymbol{x_0})\}.$$

The classifier of $y_0 = j$ is determine by the maximum value in $\hat{p_0}$:

$$\hat{y_0} = \underset{j}{\text{argmax}}\{\hat{p}_{0,j}\}.$$

Algorithm 1: k- Nearest Neighbors

Input: $D = \{\boldsymbol{x_i}, y_i\}_{i=1}^{m}$: training data;

$D_0 = \{\boldsymbol{x_0}, y_0\}$: a test observation;

$dist$: distance metric;

k: number of nearest neighbors.

Process:

1. $dist(\boldsymbol{x_i}, \boldsymbol{x_0}), i = 1, \ldots, m$
2. $dist_{sorted}(dist(\boldsymbol{x_i}, \boldsymbol{x_0})) =\uparrow dist(\boldsymbol{x_i}, \boldsymbol{x_0})$
3. $d_{(i)}{}_{i=1}^{k} \subseteq dist_{sorted}(dist(\boldsymbol{x_i}, \boldsymbol{x_0})_{i=1}^{m})$
4. $\hat{p}_{0,j} = \frac{1}{k} \sum_{i=1}^{k} I_j(y_{[i]})$
5. $\hat{y_0} = \underset{j}{\operatorname{argmax}}\{\hat{p}_{0,j}\}$

return $\hat{y_0}$.

3.1.3 Support Vector Machine (SVM)

The support vector machine (SVM) is a classifier that is obtained by expanding the feature space in a particular manner, through the application of kernels. This expansion is an extension of the original support vector classifier (SVC). SVM is a supervised machine learning technique that is utilized for classification and regression analysis.

In order to employ SVM as a base learner in ensemble methods, the model must first be trained using an appropriate kernel function, such as a linear, polynomial, or radial basis function (RBF) kernel. Cross-validation can then be used to fine-tune the SVM model and determine the hyperparameters that maximize the model's performance.

SVM is a machine learning technique that can be utilized for binary classification by identifying the hyperplane that best divides the data into two classes (James et al. 2013). The hyperplane is a flat affine $(p - 1)$ dimensional subspace in the p dimensional space.

Consider a point $X = (x_1, x_2, \ldots, x_p)^T$ is on the hyperplane H then

$$H(X) = \beta_0 + \beta^T X = 0$$

where β is a unit vector $\|\beta\| = 1$ and β_0 is a biased term.

In SVM, the hyperplane that maximizes the margin is considered as the optimal hyperplane. The margin represents the distance between the hyperplane and the nearest data points from each class. The distance between a given data point X_i and the hyperplane can be calculated as follows:

$$dist = \frac{\mid \beta_0 + \beta^T X_i \mid}{\|\beta\|}.$$

Consider the dataset $D = \{X_i, y_i\}_{i=1}^m$. In a binary classification problem with two classes $\{-1, 1\}$, the nearest points on one side of the hyperplane will be assigned to the class -1, and the points on the other side will be assigned to the class 1 (Hastie et al. 2009). The point X_i can be on the hyperplane that has class either -1 or 1. The hyperplane that contains X_i can be

$$H_1(X_i) = \beta_0 + \beta^T X_i = 1$$

or

$$H_2(X_i) = \beta_0 + \beta^T X_i = -1.$$

The margin is defined as

$$M = \frac{1}{\|\beta\|}.$$

.

For the hard margin in SVC which means that each data point must lie on the correct side of the margin, the solution for the optimization problem is the maximal margin hyperplane. The problem turns to a minimization problem as follows

$$\begin{aligned} &\underset{\beta_0,\beta}{\text{minimize}} \ \|\beta\| \\ &\text{subject to } y_i(\beta_0 + \beta^T X_i) \geq 1, i = \overline{1, m}. \end{aligned}$$

Otherwise, for the soft margin, wrong sides are allowed to lie on the wrong side of the margin. The slack variables are considered $\epsilon = (\epsilon_1, \ldots, \epsilon_m)$, when the data point is on the correct side then $\epsilon_i = 0$, hence, the total proportion amount by which the predictions are on the wrong side of their margin is $\sum_{i=1}^{m} \epsilon_i$. The constraint turns to

$$y_i(\beta_0 + \beta^T X_i) \geq (1 - \epsilon_i), \forall i$$

where $\epsilon_i \geq 0$, $\sum_{i=1}^{m} \epsilon_i \leq C$, and $C > 0$ is the "cost" parameter. In case $C = \infty$ and two classes are linearly separable, $\sum_{i=1}^{m} \epsilon_i = 0$.

The misclassification occurs when $\epsilon_i > 1$, then we want to minimize $\sum_{i=1}^{m} \epsilon_i$. The optimization problem can be expressed as

$$\begin{aligned} \underset{\beta_0, \beta, \epsilon_i}{\text{minimize}} \ & \frac{1}{2}\|\beta\|^2 + C\sum_{i=1}^{m} \epsilon_i \\ \text{subject to } & y_i(\beta_0 + \beta X_i) \geq 1 - \epsilon_i, \\ & \epsilon_i \geq 0, \\ & \forall i \in \{1, \ldots, m\}. \end{aligned}$$

This is a convex optimization problem since the norm and the linear function are convex. When C is relatively small, the sum of slack variables in the optimization problem has a minor influence on the solution, and the primary objective is to minimize the square of the norm of the weight vector β. This leads to a higher value of the sum of slack variables $\sum_{i}^{m} \epsilon_i$. Conversely, if C is large, the optimization problem prioritizes minimizing the sum of slack variables $\sum_{i}^{m} \epsilon_i$.

The Lagrangian (primal) function is

$$L_p = \frac{1}{2}\|\beta\|^2 + C\sum_{i=1}^{m} \epsilon_i - \sum_{i=1}^{m} \alpha_i[y_i(\beta_0 + \beta^T X_i) - (1 - \epsilon_i)] - \sum_{i=1}^{m} \mu_i \epsilon_i, \forall i$$

where α_i, μ_i are Lagrange multipliers for training example i.

The necessary conditions to derive the critical points are

$$\nabla_{\beta} L_p = \beta - \sum_{i=1}^{m} \alpha_i y_i X_i = 0,$$
$$\nabla_{\beta_0} L_p = -\sum_{i=1}^{m} \alpha_i y_i = 0,$$
$$\nabla_{\epsilon_i} L_p = C - \sum_{i=1}^{m} \alpha_i - \sum_{i=1}^{m} \mu_i = 0,$$

which derives

$$\beta = \sum_{i=1}^{m} \alpha_i y_i X_i,$$
$$\sum_{i=1}^{m} \alpha_i y_i = 0,$$
$$\sum_{i=1}^{m} \alpha_i = C - \sum_{i=1}^{m} \mu_i.$$

The problem turns to maximize the dual function which is

$$L_D = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j X_i^T X_j$$
$$\text{subject to} \sum_{i=1}^{m} \alpha_i y_i = 0,$$
$$0 \leq \alpha_i \leq C.$$

In addition, the approximation of β and β_0 are

$$\hat{\beta} = \sum_{i=1}^{m} \hat{\alpha}_i y_i X_i,\ \hat{\beta}_0 = y_i - \hat{\beta}^T X_i.$$

The kernel function in SVM can be used to replace the inner product between feature vectors. This function computes the inner product in a higher-dimensional feature space without actually calculating the data's coordinates in that space. This is called the kernel trick and enables the application of non-linear decision boundaries

in SVM.

$$\begin{aligned} &\text{maximize} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j K(X_i, X_j) \\ &\text{subject to} \sum_{i=1}^{m} \alpha_i y_i = 0, \\ &0 \leq \alpha_i \leq C. \end{aligned}$$

Once the SVM model is trained, it can be used to make predictions on new data points

$$\hat{y}_i = sign(\hat{\beta}^T X_i + \hat{\beta}_0)$$

where $\hat{y}_i \in \{-1, 1\}$.

3.1.4 Decision Tree

A decision tree is a supervised learning method that is a visual representation of a flow chart that resembles a tree-like structure consisting of decisions and their corresponding potential outcomes. Decision trees can be broadly classified into two types, namely regression trees and classification trees. A regression tree is used when the outcome or predicted results are continuous variables, whereas a classification tree is utilized when the outcome is a categorical or discrete variable (Suthaharan 2015). Each internal node in the tree signifies a test conducted on a specific attribute, while each branch of the tree represents an outcome of the test. The leaf nodes of the tree signify the class labels or class distribution.

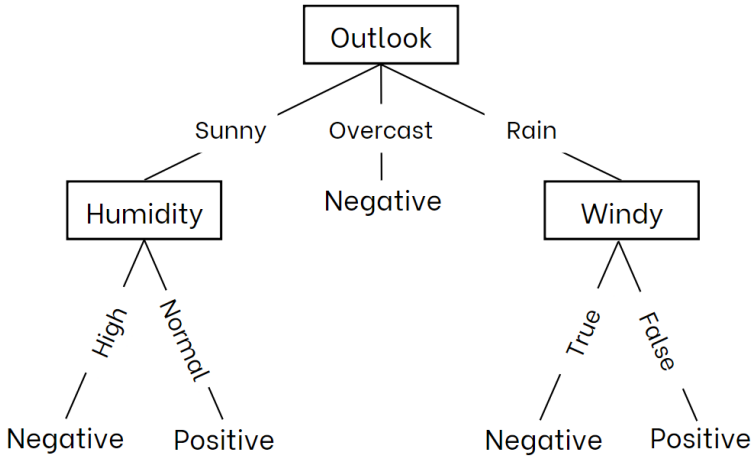


Figure 3.1: A simple decision tree.

The decision tree is generated based on the training data with a set of input features (Hambali et al. 2019). It starts by checking if a stopping condition is met, and if it is, a leaf node with the classification of the sample is returned. If the stopping condition is not met, a root node is created, and the best split condition is found for the sample and features. The possible outcomes of the split condition are stored, and for each outcome, a new sample is created with samples that satisfy the split condition. The decision tree generation function is recursively applied to the new samples, and the resulting child nodes are added to the root node. Finally, the root node of the decision tree is returned.

Algorithm 2: Decision Tree

Input: $D = \{\boldsymbol{x_i}, y_i\}_{i=1}^{m}$;

$\boldsymbol{x_i} = x_{i1}, x_{i2}, \ldots, x_{in}$;

$F = f_1, f_2, \ldots, f_n = \{f_j\}_1^n$ are the features;

Splitting criterion.

Process:

Initialize: f_j to split the data based on a selected splitting criterion.

$GenerateDecisionTree(D, F)$:

if *the stopping condition is met* **then**

- $leaf = CreateNode()$
- $leaf.label = classify(D)$
- **return** $leaf$

else

- $root = CreateNode()$
- $root.test_condition = findBestSplit(D, F)$
- $V = \{v \mid v$ a possible outcome of $root.test_condition\}$
- **for** v ***in*** V **do**
  - $D_v = \{d \mid root.test_condition(d) = v, d \in D\}$
  - $child = GenerateDecisionTree(D_v, F)$
  - Create a new child node and make it a descendant of the root node
  - Label the edge connecting the root and child nodes as v
- **return** $root$

In decision tree classification modeling, quantitative measures are necessary in two instances: firstly, to measure the information gain obtained from a feature split across data domains or subdomains, and secondly, to determine the importance of the class difference at each node to determine whether further splitting is required. Entropy, Gini Impurity, and Information Gain are effective measures for the former requirement, while class proportions, count differences, and probability measures such as ratios and percentages are useful for the latter requirement ("Induction of decision trees" - Quinlan 1986).

Assume that there are two classes in the classification model, then an arbitrary

object belongs to positive class (P) with probability $\frac{p}{p+n}$, and negative class (N) with probability $\frac{n}{p+n}$.

1. Information gain:

   Information entropy is a statistical measure that quantifies the level of uncertainty or randomness associated with a random variable. In decision tree, entropy is utilized as a metric to evaluate the impurity or randomness of the data at a particular node.

   For two classes, the information entropy is computed as follows:

$$I(p,n) = -\frac{p}{p+n}\log_2\frac{p}{p+n} - \frac{n}{p+n}\log_2\frac{n}{p+n}.$$

   In general, there are C classes then the information entropy is

$$I(p,n) = -\sum_{i=1}^{C} p_i \log_2 p_i$$

   where p_i is the probability that an arbitrary object belongs to class C_i.

   If the attribute $F = \{F_1, F_2, \ldots, F_v\}$ is the root nodes of the decision tree, and divides the classes into $O = \{O_1, O_2, \ldots, O_v\}$ where O_i contains the objects of O that have the value F_i for attribute F. The weighted average of the expected information required when F is the root node is

$$E(F) = \sum_{i=1}^{v} \frac{p_i + n_i}{p+n} I(p_i, n_i)$$

   where $I(p_i, n_i)$ is the information entropy of O_i with the number of positive classes is p_i and the number of negative classes is n_i.

   The information gain when the root node is F is

$$Gain(F) = I(p,n) - E(F).$$

2. Entropy and Cross-Entropy:

Entropy is a measure that considers the relative frequency of events. For instance, in the case of Vietnam, if there are 350 days in a year where the temperature exceeds 22° C, it can be concluded that Vietnam has a tropical climate:

$$Entropy = -\sum_{i=1}^{C} p_i \log_2 p_i$$

where C is the number of classes, and p_i is the probability of ith event

Cross-Entropy measures the difference between two probability distributions which can be the prediction probability distribution and the true distribution of outcomes. While Entropy only considers a single distribution. Assuming that there are two probability distributions $p(x)$ and $q(x)$ over the same space then

$$H(p, q) = -\sum_{i=1}^{C} p_i \log_2 q_i.$$

Note that $H(p, q) \neq H(q, p)$.

3. Gini Impurity

The Gini impurity is an alternative measure that can be utilized to assess the level of misclassification by aligning patterns with incorrect labels:

$$Gini = \sum_{i=1}^{C} p_i(1 - p_i)$$

where C is the number of classes, and p_i is the probability of ith event.

3.1.5 Neural Network

Neural network is a type of machine learning algorithm that imitates the human brain's structure and behavior. It comprises interconnected neurons arranged in several layers that process input data and generate output predictions. The network's architecture and parameters are determined from the training data through an optimization algorithm, such as stochastic gradient descent.

A neuron is a fundamental component of a neural network that takes input data with p variables $X = \{X_1, \ldots, X_p\}$ and performs a linear transformation followed

by a non-linear activation function g. Consider single layer neural network (James et al. 2013) as follows:

$$\begin{aligned} f(X) &= \beta_0 + \sum_{k=1}^{K} \beta_k h_k(X) \\ &= \beta_0 + \sum_{k=1}^{K} \beta_k g(\omega_{k0} + \sum_{j=1}^{p} \omega_{kj} X_j) \end{aligned}$$

where

- K be the number of units in hidden layers,
- $A_k = h_k(X) = g(\omega_{k0} + \sum_{j=1}^{p} \omega_{kj} X_j)$,
- $g(x)$ be an activation function,
- β_0 and ω_{k0} be the bias terms that need to be estimated,
- β_k and ω_{kj} be the weights that require estimation.

An example of single layer neural network with $K = 5$ and $p = 4$:

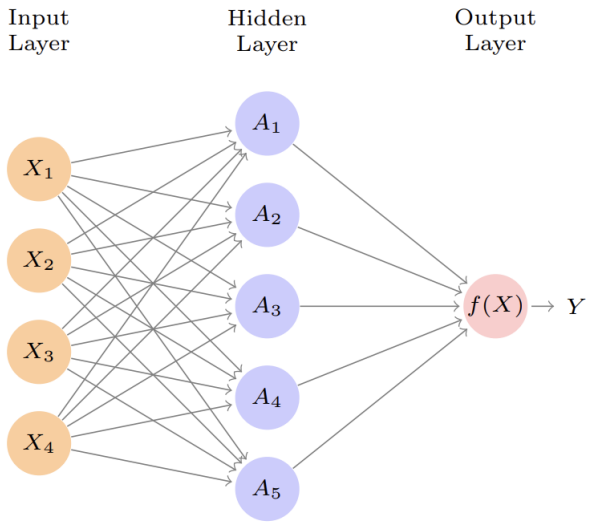


Figure 3.2: The visualization of a single-layer neural network.

Regarding multi-layer neural networks, the output of the previous hidden layer will become the input for the current layer. Suppose that the number of layers in the neural networks is 4, and R be the number of units in the output layer.

- Input to the first hidden layer (1st layer to 2nd layer):

$$A_k^{(1)} = h_k^{(1)}(X) = g\left(\omega_{k0}^{(1)} + \sum_{j=1}^{p} \omega_{kj}^{(1)} X_j\right), \forall k = 1, \ldots, K_1.$$

- Hidden layer to hidden layer (2nd layer to 3rd layer):

$$A_l^{(2)} = h_l^{(2)}(X) = g\left(\omega_{l0}^{(2)} + \sum_{k=1}^{K_1} \omega_{lk}^{(2)} A_k^{(1)}\right), \forall l = 1, \ldots, K_2.$$

- Hidden layer to output layer (3rd layer to 4th layer):

$$f_m(X) = \beta_{m0} + \sum_{l=1}^{K_2} \beta_{ml} h_l^{(2)}(X) = \beta_{m0} + \sum_{l=1}^{K_2} \beta_{ml} A_l^{(2)}$$

  for $m = 1, \ldots, R$.

There exist several frequently used activation functions, such as the ReLU, sigmoid.

1. Rectified Linear Unit (ReLU) is regularly utilized in hidden layers:

$$g(z) = z_+ = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{otherwise} \end{cases}$$

2. Sigmoid function is normally applied for the output layer in classification problem:

$$g(z) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}.$$

The neural network learns the weights and biases of its neurons by minimizing a cost function that quantifies the error between the predicted and true outputs. Backpropagation algorithm is often used for computing the gradients of the cost function with respect to the weights and biases, which are then updated using an optimization algorithm.

To improve the performance of neural network, various techniques such as regularization, dropout, and batch normalization can be used to prevent overfitting and

improve generalization. Additionally, neural network can be customized by selecting appropriate activation functions, number of layers, and number of neurons per layer.

3.2 Stacking Algorithms

A stacking ensemble method is a technique that combines multiple predictive models to derive a more accurate and robust model. This involves using a meta-model that takes as input the predictions of several base models, which may be different types of algorithms or trained on different subsets of the data. The meta-model then learns to weight the predictions of the base models to produce the final output.

A meta model is a model that learns how to combine the predictions of multiple other models (known as base models) to improve the overall prediction accuracy. The idea behind a meta model is that different base models may excel in different parts of the data space, and by combining them, we can create a more accurate and robust model.

This illustrates how stacking can be executed using logistic regression, neural network, and decision tree, with a logistic regression meta-model being employed.

1. Train individual base models:
    - Train a logistic regression model on the training data and obtain its predictions for the validation and test sets.
    - Train a neural network model on the training data and obtain its predictions for the validation and test sets.
    - Train a decision tree model on the training data and obtain its predictions for the validation and test sets.
2. Combine the predictions from the base models:
    - Concatenate the predictions from the logistic regression, neural network, and decision tree models to form a new feature matrix for the validation and test sets. This creates a new set of features where each column corresponds to a different base model.

- Use the new feature matrix as input to train the meta-model.
- Use the meta-model to make predictions on the validation and test sets.

3. Evaluate the performance of the stacked model:
   - Compute the accuracy, precision, recall, F1 score, or other metrics to evaluate the performance of the stacked model on the validation and test sets.
   - Compare the performance of the stacked model to that of the individual base models to determine if stacking has improved the predictive accuracy of the model.

3.3 Bagging-based Ensemble Algorithms

Bagging, or bootstrap aggregating, is a machine learning technique that aims to improve the accuracy and stability of prediction models by using multiple subsets of the original data for training. This approach involves creating new subsets of the data by randomly sampling from the original dataset, building models on each of these new subsets, and then aggregating their predictions. There are several machine learning algorithms that can be used for bagging method, such as random forest, and bagged neural network. In addition, the machine learning which supports sampling with replacement can use for this approach.

An illustration of using bagging with logistic regression, neural network, and decision trees as base learners and the binary outcomes:

1. Split the training data into $B = 3$ bootstrap samples.
2. For each bootstrap sample $b = 1, \ldots, B$ do the following:
   (a) Train a logistic regression model $h_1^{\text{logistic}}(x)$ on the 1st bootstrap sample.
   (b) Train a neural network model $h_2^{\text{NN}}(x)$ on the 2nd bootstrap sample.
   (c) Train a decision tree model $h_3^{\text{tree}}(x)$ on the 3th bootstrap sample.

Note that the weak learners can be swapped between b-th bootstrap sample.

3. For each base learner, combine the B models trained on the respective bootstrap samples to obtain an ensemble of models.

4. Predict the class label of a new data point x using a majority vote of the predictions of the three ensembles obtained from the three base learners.

3.3.1 Random Forest

According to Cambridge dictionary, forest is a large area of land covered with trees which is the main idea of Random Forest. Regarding to classification, Random Forest algorithm collects class votes from individual decision trees and determines the final class prediction by taking the majority vote of all trees. In regression problems, the prediction for a target point in a random forest is obtained by taking the average of the predictions from each individual tree (Hastie et al. 2009).

Algorithm 3: Random Forest

Input: $D = \{\boldsymbol{x_i}, y_i\}_{i=1}^{m}$;

$\boldsymbol{x_i} = x_{i1}, x_{i2}, \ldots, x_{in}$;

$F = f_1, f_2, \ldots, f_n = \{f_j\}_{j=1}^{n}$ are the features;

Splitting criterion;

B: the number of trees to be generated.

Process:

for *b=1* ***to*** B **do**

Draw a random bootstrap sample from the training data of size m, denoted by D_b

Grow a random-forest tree $T_b = GenerateDecisionTree(D_b, F)$

return *The forest trees* $\{T_b\}_{b=1}^{B}$

To generate a prediction for a new data point $\boldsymbol{x}$:

- Regression: $\hat{f}_{rf}^{B}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(\boldsymbol{x})$.
- Classification: $\hat{C}_{rf}^{B}(x)$ = majority vote $\{\hat{C}_b(\boldsymbol{x})\}_{b=1}^{B}$ where $\hat{C}_b(\boldsymbol{x})$ be the predicted classification of bth tree in random forest.

In order to measure the accuracy of the random forest model, it is important to consider *Out-Of-Bag* samples which are samples that are not included in the boostrap sample. An out-of-bag error mentioned the proportion of *Out-Of-Bag* samples that are incorrectly classified, Moreover, it estimate is nearly equivalent to the error estimate obtained through N-fold cross-validation (Breiman 1996, Wolpert 1992).

3.3.2 Bagged Neural Network

Similar to other bagging methods, the main idea of Bagged Neural Network is a combination of bagging ensemble method and neural network. The Bagged Neural Network algorithm can be described in the following steps:

1. Generate B bootstrap samples from the original dataset.
2. For each bootstrap sample, train a neural network on the data.
3. Use the trained neural network to predict the class label of new data points.
4. Aggregate the predictions of all neural networks using majority voting.

Algorithm 4: Bagged Neural Network

Input: $D = \{\boldsymbol{x_i}, y_i\}_{i=1}^{m}$;

$\boldsymbol{x_i} = x_{i1}, x_{i2}, \ldots, x_{in}$;

$F = f_1, f_2, \ldots, f_n = \{f_j\}_1^n$ are the features;

Given a new input data point $\boldsymbol{x}$.

Process:

for *b=1* ***to*** B **do**

$f_b = \underset{f_b}{\operatorname{argmin}} \sum_{(\boldsymbol{x}_j, y_j), D_b} L[(f_b(\boldsymbol{x_j}), y_j]$

$\hat{y}_b(\boldsymbol{x}) = f_b(\boldsymbol{x})$

return $\hat{y}(\boldsymbol{x}) = mode(\{\hat{y}_b(\boldsymbol{x})\}_{b=1}^{B})$

Commonly used evaluation metrics for Bagged Neural Network include techniques like cross-validation, as well as metrics such as average error across all repetitions and out-of-bag error.

3.4 Boosting Algorithms

The boosting technique is an ensemble learning technique that involves training multiple base learners in sequence. The subsequent learners try to correct the mistakes of the previous ones. The final model is a combination of all base learners weighted by their performance.

For example, the algorithm for gradient boosting with logistic regression, neural network, and decision tree can be outlined as follows:

1. Randomly select 3 subsets from a given training dataset.
2. Train the first subset by using the weak learner - logistic regression.
3. After training the weak learner, evaluate it on the training data, which may lead to incorrect predictions for some data points.
4. Every data point that is incorrectly predicted is utilized to revise the second subset of data.
5. Train and test for the second weak learner - neural network on the updated subset.
6. Similarly, the incorrectly predicted data point will be updated to the third subset.
7. The third weak learner - decision tree will be applied for training and testing on previous updated subset.
8. Acquire the final prediction.

Note that the order of using weak learners can be changed.

This algorithm employs three types of base learners, namely logistic regression, neural network, and decision trees. During each iteration, the negative gradient of the loss function is used to train these three models, and their outputs are merged using a step size determined by minimizing the loss function. The final model is a composite of all the models produced at each iteration.

Boosting algorithms are frequently utilized in machine learning, and some of the popular ones include Adaptive Boosting, Gradient Boosting, Extreme Gradient Boosting, and Light Gradient Boosting Machine.

3.4.1 Adaptive Boosting (AdaBoost)

The AdaBoost algorithm, first introduced by Freund, Robert E Schapire, et al. (1996), is a machine learning technique that has been widely used and modified in various contexts. Its key idea is to adjust the weights of misclassified instances in the training data to create a new model from the previous weak learners. By focusing on the misclassified examples, the algorithm allows the new learner to learn and improve its performance. The final model is computed by summing the weighted contributions of individual weak learners, with each learner's weight determined by its accuracy on the training set.

In this thesis, there is a focus on a commonly used ensemble learning algorithm for binary classification, where the label y_i takes values in $\{-1, 1\}$ (Zhou 2012, Robert E. Schapire 2013).

Input:

- Data set $D = \{(\boldsymbol{x_1}, y_1), (\boldsymbol{x_2}, y_2), \ldots, (\boldsymbol{x_m}, y_m)\} = \{\boldsymbol{x_i}, y_i\}_{i=1}^{m}$

  where

  $\boldsymbol{\chi}$ is some domain that $\boldsymbol{x_i} \in \boldsymbol{\chi}$;

  $\boldsymbol{x_i}$ has n attributes;

  m be the number of training examples;

  $y_i \in \{-1, 1\}$ be the binary classifiers.

- The base learning algorithm $\mathfrak{L}$.

- T be the number of learning rounds.

Process:

1. Initialize the weight: $\omega_1(i) = 1/m$ for $i = 1, \ldots, m$.

2. For $t = 1, \ldots, T$:

   - The weak learner is trained by the distribution ω_t to derive the classifiers $h_t = \mathfrak{L}(D, \omega_t)$.
   - The weak hypothesis is $h_t : \boldsymbol{\chi} \to \{-1, 1\}$.
   - The aim is to choose h_t that has low weighted error as follows:

$$\begin{aligned}\epsilon_t &= \frac{1}{2}\sum_{i=1}^{m} \omega_t(i)(1 - y_i h_t(\boldsymbol{x_i})) \\ &= \sum_{i=1}^{m} \omega_t(i)\mathbb{I}(h_t(\boldsymbol{x_i}) \neq y_i) \\ &= P_{i\sim\omega_t}[h_t(\boldsymbol{x_i}) \neq y_i].\end{aligned}$$

   - The weight of h_t is determined as

$$\alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right).$$

   - Updating the weight distribution

$$\omega_{t+1}(i) = \frac{\omega_t(i)\exp(-\alpha_t y_i h_t(\boldsymbol{x_i}))}{Z_t}$$

     where Z_t is the normalization factor that makes ω_{t+1} as a distribution.

Output the final classifier:

$$H(\boldsymbol{x}) = sign\left(\sum_{t=1}^{T} \alpha_t h_t(\boldsymbol{x})\right).$$

Since the labels $y_i \in \{-1, 1\}$ is considered, the sign of the final classifier by using additive weighted combination of weak learners is as follows:

$$F(\boldsymbol{x}) = \sum_{t=1}^{T} \alpha_t h_t(\boldsymbol{x}).$$

In order to find the optimal weight (α_t), Freund, Robert E Schapire, et al. (1996)

considered a loss function, the exponential loss was considered:

$$\begin{aligned} l_{exp} &= \mathbb{E}_{i\sim\omega_t}[e^{-y_i\alpha_t h_t(\boldsymbol{x_i})}] \\ &= \mathbb{E}_{i\sim\omega_t}[e^{-\alpha_t}\mathbb{I}(h_t(\boldsymbol{x_i}) = y_i) + e^{\alpha_t}\mathbb{I}(h_t(\boldsymbol{x_i}) \neq y_i)] \\ &= e^{-\alpha_t}P_{i\sim\omega_t}[h_t(\boldsymbol{x_i}) = y_i] + e^{\alpha_t}P_{i\sim\omega_t}[h_t(\boldsymbol{x_i}) \neq y_i] \\ &= e^{-\alpha_t}(1-\epsilon_t) + e^{\alpha_t}\epsilon_t. \end{aligned}$$

Since

$$\sum_{i=1}^{m}\omega_t(i) = P_{i\sim\omega_t}[h_t(\boldsymbol{x_i}) \neq y_i] + P_{i\sim\omega_t}[h_t(\boldsymbol{x_i}) = y_i] = 1,$$

this implies

$$P_{i\sim\omega_t}[h_t(\boldsymbol{x_i}) = y_i] = 1-\epsilon_t.$$

To derive the optimal value for α_t to minimize the loss function, let take the partial derivative corresponds to α_t as follows:

$$\frac{\partial l_{exp}}{\partial\alpha_t} = -e^{-\alpha_t}(1-\epsilon_t) + e^{\alpha_t}\epsilon_t = 0.$$

Then the solution is

$$\alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right).$$

Algorithm 5: AdaBoost

Input: $D = \{\boldsymbol{x_i}, y_i\}_{i=1}^{m}$;

$\mathfrak{L}$: base learning algorithm;

T: number of iterations.

Process:

Initialize: $\omega_1(i) = 1/m$ for $i = 1, \ldots, m$.

for $t = 1$ ***to*** T **do**

$h_t = \mathfrak{L}(D, \omega_t)$

$\epsilon_t = P_{i \sim \omega_t}[h_t(\boldsymbol{x_i}) \neq y_i]$

if $\epsilon_t > 0.5$ **then**

break

$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$

$\omega_{t+1}(i) = \frac{\omega_t(i) \exp(-\alpha_t y_i h_t(\boldsymbol{x_i}))}{Z_t}$

return $H(\boldsymbol{x}) = sign\left(\sum_{t=1}^{T} \alpha_t h_t(\boldsymbol{x})\right)$

3.4.2 Gradient Boosting (GBoost)

Gradient Boosting is a popular machine learning technique for solving regression, classification, and ranking problems (Friedman 2001). It works by combining multiple weak learners to create a powerful predictive model. This is done in a sequential manner, with each new weak learner being trained based on the errors made by the previous one. In each iteration, the new model is built by fitting it to the negative gradient of the loss function, using the output of the previous model. The model objective is to minimize the loss function and achieve a high level of accuracy.

Input:

- Data set $D = \{(\boldsymbol{x_1}, y_1), (\boldsymbol{x_2}, y_2), \ldots, (\boldsymbol{x_m}, y_m)\} = \{\boldsymbol{x_i}, y_i\}_{i=1}^{m}$

  where

  $\boldsymbol{x_i}$ has n attributes;

  m be the number of training examples;

  $y_i \in \{0, 1\}$ be the binary classification.

- The loss function: $L(y, F(\boldsymbol{x}))$

  where $F(\boldsymbol{x})$ represents the model's predicted output for a given input $\boldsymbol{x}$.

  $F^*(\boldsymbol{x})$ is a function that maps $\boldsymbol{x}$ to y to minimize the loss function:

$$F^* = \underset{F}{\operatorname{argmin}}(E_{y,\boldsymbol{x}} L(y, F(\boldsymbol{x})) = \underset{F}{\operatorname{argmin}}(E_{\boldsymbol{x}}[E_y(L(y, F(\boldsymbol{x}))) \mid \boldsymbol{x}].$$

  The "additive" expansion form is mentioned by Freund, Robert E Schapire, et al. (1996).

$$F\left(\boldsymbol{x}, \{\beta_k, \boldsymbol{a}_k\}_1^K\right) = \sum_{k=1}^{K} \beta_k h(\boldsymbol{x}, \boldsymbol{a}_k)$$

  where

  K be the number of iterations;

  $\{\beta_k\}_0^K$ be the set of weights applied to the residuals at each iteration of the algorithm. These weights determine the degree to which the residuals are corrected by the addition of each subsequent weak learner;

  $h(\boldsymbol{x}, \boldsymbol{a}_k)$ be a weak learner;

  $\boldsymbol{a} = \{a_1, a_2, \ldots, a_k\}$ represent the expansion coefficients assigned to the weak learners in the additive expansion which control the contribution of each weak learners to the final boosted model.

Process:

1. Initialize $F_0(\boldsymbol{x}) = \underset{\rho}{\operatorname{argmin}} \sum_{i=1}^{m} L(y_i, \rho)$.
2. For $k = 1, \ldots, K$:
   - Compute the "pseudo" residual or the steepest descent:

$$\tilde{y_i} = -\left[\frac{\partial L(y_i, F(\boldsymbol{x_i}))}{\partial F(\boldsymbol{x_i})}\right]_{F(\boldsymbol{x})=F_{k-1}(\boldsymbol{x})}, i = \overline{1, m}.$$

   - The function $h(\boldsymbol{x}, \boldsymbol{a})$ is fit by least-squares:

$$\boldsymbol{a}_k = \underset{a,\beta}{\operatorname{argmin}} \sum_{i=1}^{m} [\tilde{y_i} - \beta h(\boldsymbol{x_i}, \boldsymbol{a})]^2$$

where ρ is the step size at each iteration of the gradient descent.

- ρ_k is multiplied with the gradient of the loss function to update the predictions of the model in each iteration. It determines the degree to which the contribution of each weak learner is shrunk towards zero:

$$\rho_k = \underset{\rho}{\operatorname{argmin}} \sum_{i=1}^{m} L(y_i, F_{k-1}(\boldsymbol{x_i}) + \rho h(\boldsymbol{x_i}, \boldsymbol{a}_k)).$$

- The approximation updated:

$$F_k(\boldsymbol{x}) = F_{k-1}(\boldsymbol{x}) + \rho_k h(\boldsymbol{x}, \boldsymbol{a}_k).$$

Algorithm 6: Gradient Boosting

Input: $D = \{\boldsymbol{x_i}, y_i\}_{i=1}^{m}$;

$L(y, F(\boldsymbol{x}))$: loss function;

M: number of weak learners to be added to the model.

Process:

Initialize: $F_0(\boldsymbol{x}) = \underset{\rho}{\operatorname{argmin}} \sum_{i=1}^{m} L(y_i, \rho)$.

for $k = 1$ ***to*** K **do**

$\tilde{y_i} = -\left[\frac{\partial L(y_i, F(\boldsymbol{x_i}))}{\partial F(\boldsymbol{x_i})}\right]_{F(\boldsymbol{x})=F_{k-1}(\boldsymbol{x})}, i = \{1, \ldots, m\}$

$\boldsymbol{a}_k = \underset{\boldsymbol{a},\beta}{\operatorname{argmin}} \sum_{i=1}^{m} [\tilde{y_i} - \beta h(\boldsymbol{x_i}, \boldsymbol{a})]^2$

$\rho_k = \underset{\rho}{\operatorname{argmin}} \sum_{i=1}^{m} L(y_i, F_{k-1}(\boldsymbol{x_i}) + \rho h(\boldsymbol{x_i}, \boldsymbol{a}_k))$

$F_k(\boldsymbol{x}) = F_{k-1}(\boldsymbol{x}) + \rho_k h(\boldsymbol{x}, \boldsymbol{a}_k)$

return $F_K(\boldsymbol{x})$

3.4.3 Extreme Gradient Boosting (XGBoost)

XGBoost is a supervised learning algorithm that uses an ensemble of decision trees to make predictions. One of the original papers on extreme gradient boosting is "Xgboost: A scalable tree boosting system" by T. Chen and Guestrin (2016). XGBoost uses a greedy algorithm to build a sequence of decision trees in an iterative manner. At each step, it chooses the best split to minimize the loss function. This process continues until a stopping criterion is met, such as reaching the maximum

number of trees or the minimum improvement in loss reduction. The resulting ensemble of decision trees is then used to make predictions. The goal of this algorithm is to optimize the cost objective function as follows:

$$loss = \sum_{i=1}^{m} l(y_i, \hat{y_i}^{(t)}) + \sum_{i=1}^{t} \Omega(f_i)$$

where

$\sum_{i=1}^{m} l(y_i, \hat{y_i}^{(t)})$ be the empirical loss function;

$\Omega(f)$ be the regularization term which is expressed as

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \tag{3.5}$$

where T be the number of leaf nodes, w_j be the value on each leaf node, γ, λ be the coefficients.

Let $\hat{y}_i^{(t)}$ be the predictive value of the model at t^{th}. Initialize $\hat{y}_i^{(0)} = 0$, then at time t the outcome is computed as follows:

$$\hat{y}_i^{(t)} = \sum_{j=1}^{t} f_j(\boldsymbol{x}_i) = \hat{y}_i^{(t-1)} + f_t(\boldsymbol{x}_i),$$

$$\hat{y}_i = \hat{y}_i^{(T)} = \sum_{t=1}^{T} f_t(\boldsymbol{x}_i).$$

The tree selected at each step is the one that minimizes the objective function by replacing $\hat{y}_i^{(t)}$ in the objective function at t^{th}:

$$loss = \sum_{i=1}^{m} l(y_i, \hat{y_i}^{(t-1)} + f_t(\boldsymbol{x}_i)) + \sum_{i=1}^{t-1} \Omega(f_i) + \Omega(f_t)$$

where

$\sum_{i=1}^{t-1} \Omega(f_i)$ is the regular term of $t-1$ term.

$\Omega(f_t)$ is the regular term of t term.

A simplified expression for the second-order Taylor expansion at $\hat{y_i}^{(t-1)}$ can be obtained by taking $\hat{y_i}^{(t-1)} + f_t(\boldsymbol{x}_i)$ in the error function as a whole:

$$l\left(y_i, \hat{y}^{(t-1)} + f_t(\boldsymbol{x}_i)\right) \approx l\left(y_i, \hat{y}^{(t-1)}\right) + g_i f_t(\boldsymbol{x}_i) + \frac{1}{2} h_i f_t(\boldsymbol{x}_i)^2$$

where

g_i is the first derivative of the loss function evaluated at $\hat{y_i}^{(t-1)}$;

h_i is the second derivative of the loss function evaluated at $\hat{y_i}^{(t-1)}$.

This simplified expression is then used to update the estimate of the function by finding the optimal value of $f_t(\boldsymbol{x}_i)$ that minimizes the objective function. This is done using a line search algorithm, which finds the value of $f_t(\boldsymbol{x}_i)$ that minimizes the quadratic approximation of the loss function.

Since $l\left(y_i, \hat{y_i}^{(t-1)}\right)$ is a constant, the objective function can reduce to

$$loss \approx \sum_{i=1}^{m} \left[g_i f_t(\boldsymbol{x}_i) + \frac{1}{2} h_i f_t^2(\boldsymbol{x}_i) \right] + \Omega(f_t). \tag{3.6}$$

Substituting (3.5) into (3.6), each x_i will fall on a leaf node, and the corresponding output of each leaf node is w_j, this yields

$$loss \approx \sum_{i=1}^{m} \left[g_i w_j + \frac{1}{2} h_i w_j^2 \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^{T} w_j^2. \tag{3.7}$$

Let $I_j = \{i \mid q(\boldsymbol{x}_i) = j\}$ as the instance set of leaf j. Then the loss function can be rewritten as:

$$loss = \sum_{j=1}^{T} \left[\left(\sum_{i \in I_j} g_i \right) \omega_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) \omega_j^2 \right] + \gamma T. \tag{3.8}$$

The optimal weight ω_j^* of leaf j is computed as:

$$\omega_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}.$$

Then

$$loss^* = -\frac{1}{2}\sum_{j=1}^{T}\frac{\left(\sum_{i\in I_j} g_i\right)^2}{\sum_{i\in I_j} h_i + \lambda} + \gamma T$$
$$= -\frac{1}{2}\sum_{j=1}^{T}\frac{G_j^2}{H_j + \lambda} + \gamma T$$

where $G_j = \sum_{i\in I_j} g_i$ and $H_j = \sum_{i\in I_j} h_i$.

Instead of using an approach that adds all possible branches to the tree, a greedy algorithm that starts with a single leaf and progressively adds branches to the tree is used. Suppose that I_L and I_R represent the sets of instances in the left and right nodes respectively after performing a split. By defining $I = I_L \bigcup I_R$, the reduction in loss that occurs due to the split can be expressed as follows:

$$loss_{split} = loss_I - (loss_{I_L} + loss_{I_R})$$
$$= \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda}\right] - \gamma$$

where $G = G_L + G_R$, and $H = H_L + H_R$.

Since applying the exact greedy algorithm for a huge dataset is costly and time consuming, T. Chen and Guestrin proposed some approximation greedy algorithms consist of weighted quantile sketch and sparsity-aware split finding. The main idea of weighted quantile sketch is to represent a data distribution by a set of weighted quantiles, which are more robust to outliers and noise than traditional summary statistics. This allows for efficient and accurate calculation of statistics and quantiles on large datasets without storing or processing the full data. In XGBoost, the weighted quantile sketch is used to approximate the distribution of gradient and Hessian values, which are used in the tree construction process. The purpose of sparsity-aware split finding is to build tree with missing data and how to deal with new observations when there is missing data.

The implementation of the XGBoost can be described as follows:

1. Initialization: $\hat{y}_i^{(0)} = 0$.

2. For $t = 1$ to T:

   (a) Compute G_i and H_i.

   (b) Build the tree $f_t(x)$:

      (A) Start with an empty tree.

      (B) For each node, find the best splitting for each feature and select the one maximizing gain.

   (c) Add the tree $f_t(x)$ to the model: $\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(\boldsymbol{x}_i)$, where η is the learning rate.

   There are some differences between traditional gradient boosting and XGBoost:

   - Regularization: XGBoost has an additional regularization term added to the loss function that helps to prevent overfitting. This regularization term penalizes large weights and reduces the complexity of the model.
   - Splitting Strategy: XGBoost uses a more advanced tree splitting strategy that considers the second-order derivative of the loss function, in addition to the first-order derivative used by traditional gradient boosting. This helps to make the algorithm more efficient and accurate.
   - Parallel Processing: XGBoost has built-in support for parallel processing, allowing it to run faster on multi-core CPUs and distributed clusters.
   - Scalability: XGBoost is designed to be highly scalable and can handle large datasets with millions of examples and thousands of features.

3.4.4 Light Gradient Boosting Machine (LightGBM)

LightGBM (Light Gradient Boosting Machine) is a gradient boosting framework that uses tree-based learning algorithms which was introduced in "LightGBM: A Highly Efficient Gradient Boosting Decision Tree" by Ke et al. (2017). It is designed to be efficient in terms of both computation time and memory usage, making it suitable for handling large-scale data sets.

The key idea behind LightGBM is to use a technique called Gradient-based One-Side Sampling (GOSS) to filter out the data instances that contribute less to the gradient calculation, thereby reducing the computation cost. Additionally, LightGBM uses Histogram-based Gradient Boosting (HGB) to speed up the training process by binning the continuous feature values into discrete values.

LightGBM has been shown to achieve state-of-the-art performance on a variety of machine learning tasks, including classification, regression, and ranking. It has been widely adopted in industry and academia due to its high efficiency and accuracy.

GOSS is an algorithm that samples instances based on their gradients and handles instances with large and small gradients differently. It sorts instances based on their gradient magnitudes and selects the top $a\%$ instances with large gradients, while randomly sampling $b\%$ instances with small gradients. To balance the bias introduced by sampling, GOSS multiplies the gradients of randomly sampled instances by a constant multiplier of $(1 - a)/b$ during information gain calculation. This approach enhances the importance of under-trained instances without significantly altering the original data distribution.

Algorithm 7: Gradient-based One-Side Sampling

Input: $D = (\boldsymbol{x_i}, y_i)_{i=1}^{m}$;

K : number of iterations;

a : sampling ratio of large gradient data;

b : sampling ratio of small gradient data;

L : loss function;

h : weak learner.

Process:

Initialize: models $\leftarrow \{\}$;

fact $\leftarrow \dfrac{1-a}{b}$;

topN $\leftarrow a \times m$;

randN $\leftarrow b \times m$;

$F_0(\boldsymbol{x}) = \underset{a,\rho}{\operatorname{argmin}} \Sigma_{i=1}^{m} L(y_i, \rho)$.

for $k = 1$ ***to*** K **do**

$g_i = -\left[\frac{\partial L(y_i, F(\boldsymbol{x_i}))}{\partial F(\boldsymbol{x_i})}\right]_{F(\boldsymbol{x})=F_{k-1}(\boldsymbol{x})}, i = \{1, \ldots, m\}$

$\omega = \{1, 1, \ldots, 1\} \in \mathbb{R}^m$

$D_{sorted} = \{(x_i, y_i)\}$ sorted by $\mid g_i \mid$

$D_{top} = D_{sorted}[1 : \text{topN}]$

$D_{rand} = \text{Random Pick}\{D_{sorted}[\text{topN} : m], \text{randN}\}$

$D_{used} = D_{top} + D_{rand}$

Assign *weight* to the small gradient data

$\omega[D_{rand}] = \omega[D_{rand}] \times \text{fact}$

$newModel = h(D_{used}, -g[D_{used}], w[D_{used}]))$

models.append(newModel)

To reduce the number of features in high-dimensional sparse data, exclusive features that never take nonzero values together can be bundled together nearly losslessly. This is achieved by using a feature scanning algorithm to build feature histograms from the feature bundles in the same way as individual features.

Algorithm 8: Greedy Bundling

Input: F : features;

K : max conflict count;

Construct graph G;

searchOrder $\leftarrow$ G.sortByDegree();

bundles $\leftarrow$ {};

bundlesConflict $\leftarrow$ {}.

Process:

```
for i in searchOrder do
    needNew ← True
    for j = 1 to len(bundles) do
        cnt ← ConflictCnt(bundles[j], F[i])
        if cnt + bundlesConflict[i] ≤ K then
            bundles[j].add(F[i]), needNew ← False
            break
    if needNew then
        Add F[i] as a new bundle to bundles
return bundles
```

Algorithm 9: Merge Exclusive Features

Input: $numData$: number of data;

F : one bundle of exclusive features;

$binRanges \leftarrow \{0\}$;

$totalBin \leftarrow 0$.

Process:

for f ***in*** F **do**
    $totalBin+ = f.numBin$
    $binRanges.append(totalBin)$

$newBin \leftarrow newBin(numData)$

for i = *1* ***to*** $numData$ **do**
    $newBin[i] \leftarrow 0$
    for j = *1* ***to*** *len(F)* **do**
        if *F[j].bin[i]* $\neq$ *0* **then**
            $newBin[i] \leftarrow F[j].bin[i] + binRanges[j]$

return $newBin, binRanges$

3.4.5 Summary

Stacking, bagging, and boosting are all ensemble learning techniques used in machine learning to improve the predictive performance of models. However, they differ in their approach to combining multiple base learners to form a final model.

Bagging involves training multiple base learners independently on different subsets of the data, and then aggregating their predictions to form the final model. Each base learner has an equal vote in the final prediction. This helps to reduce the variance of the model and improve its robustness.

Boosting, on the other hand, involves training multiple base learners sequentially, where each subsequent learner tries to correct the mistakes of its predecessor. The final model is a weighted combination of all the base learners, where the weight assigned to each learner depends on its performance on the training data. This helps to reduce the bias of the model and improve its accuracy.

Stacking involves training multiple base learners independently, and then using their predictions as input to a meta-learner, which is trained to make the final prediction. The meta-learner can be a simple linear model or a more complex model like a neural network. Stacking is designed to capture the strengths of each base learner and improve the overall performance of the model.

To sum up, bagging-based and stacking ensemble methods are used to optimize the variance of prediction result to reduce the classification prediction error, while boosting-based ensemble methods are used to reduce the bias in classification prediction by adding learners iteratively (Opitz and Maclin 1999). Ensemble modeling has been demonstrated to enhance the effectiveness of credit scoring models, particularly in instances where the data is imbalanced or the patterns are complex and non-linear (Xiao et al. 2021). However, it is important to keep in mind that ensemble modeling is not a silver bullet, and its effectiveness depends on several factors such as the choice of base models, the quality of the data, and the evaluation metrics used to assess the performance of the model.

3.5 Evaluation metrics

The study utilizes AUC-ROC as the evaluation metric, where AUC and ROC represent Area Under Curve and Receiver Operating Characteristics, respectively. This metric is particularly suitable for imbalanced datasets since it can assess the model's capability to distinguish between positive and negative classes at varying thresholds, regardless of any changes in class distribution. Similarly, the competition also employs AUC-ROC as its evaluation metric, with both Public Score and Private Score being calculated from 20% and 80% of the test data, respectively. The Private Score is used to determine the final results. In comparison to accuracy, which may be misleading when evaluating imbalanced data, AUC-ROC provides a more precise assessment of the model's performance in accurately classifying positive and negative samples. The ROC is simply a graphical representation of the True Positive Rate (TPR) versus the False Positive Rate (FPR) at all possible thresholds, while the AUC represents the entire area under this ROC curve (GeeksforGeeks 2021).

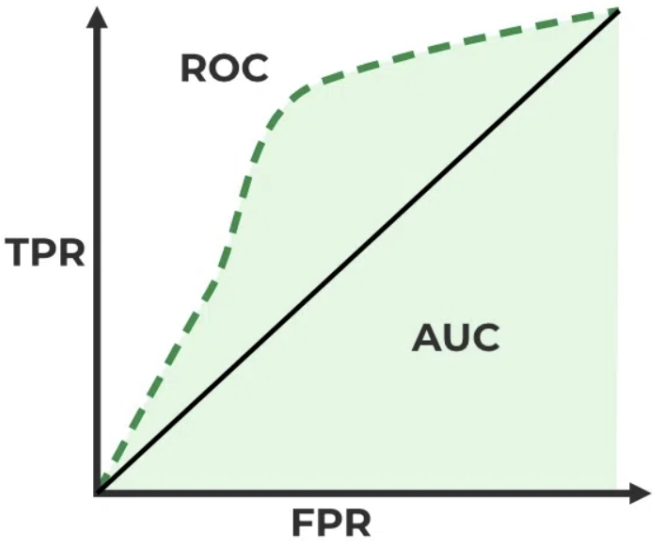


Figure 3.3: AUC-ROC Curve.

Chapter 4

Empirical Results

4.1 Explore data

The Home Credit Default Risk dataset (HomeCredit 2018) is a widely-used dataset available on Kaggle, which contains anonymized credit and demographic data for over 300,000 loan applicants, along with information about whether the loan was repaid or defaulted. A detailed description of each feature can be found on Kaggle, along with exploratory data analysis notebooks and machine learning models that have been developed using the dataset.

4.1.1 Data structure and description

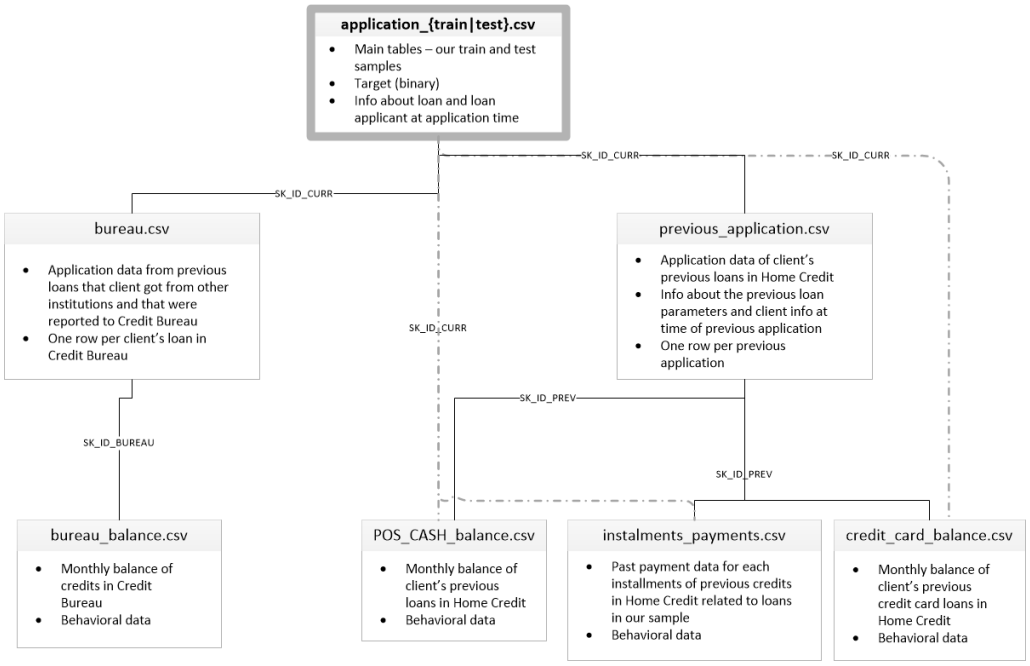


Figure 4.1: Data Structure Diagram.

The Home Credit Default Risk dataset consists of 7 data files that are interconnected by keys such as SK_ID_CURR, SK_ID_PREV, SK_ID_BUREAU, and the relationships among them are illustrated in Figure 4.1.

- *application_{train | test}.csv*

  These data files contain 122 columns of static data that provide information about the user's profile at the time of applying for the loan. The train and test files differ in that the train data include a column named "TARGET", while the test data do not.

- *previous_application.csv*

  This file includes all of previous loans of customers that are included in *application_{train| test}.csv*. Each previous loan application related to clients in our data sample is represented by one row. In addition, the data file contains 37 columns and a total of 1,670,214 rows, which is considered a large dataset. The key between *application_{train| test}.csv* and *previous_application.csv* is SK_ID_CURR.

- *installments_payments.csv*

  The file named *installments_payments.csv* contains the payment details for previous Home Credit loans that are linked to the loans in the provided sample which has 13,605,401 entries. Each row in the file represents either a single installment payment or a payment made on a prior Home Credit loan that is linked to the loans in the given sample.

- *credit_card_balance.csv*

  The presented table illustrates the monthly balance status of the applicant's previous credit cards with Home Credit. It includes a row for each month of history for every past credit card, encompassing consumer credit and cash loans. The total number of rows in the table is 3,840,312 data points, which is computed by multiplying the number of loans in the sample by the number of previous credit cards that are related to those loans and the number of months with recorded history for each credit card.

- *POS_CASH_balance.csv*

  The table depicts the monthly balance status of the applicant's previous point-of-sale (POS) transactions and cash loans with Home Credit, containing 10,001,358 entries. For each past credit, encompassing consumer credit and cash loans, there is a row for every month of history. The total number of rows in the table is calculated by multiplying the number of loans in the sample by the number of past credits that correspond to those loans, and by the number of months with recorded historical data for each previous credit.

- *bureau.csv*

  The presented table displays data on a client's prior credits, which were reported to the Credit Bureau by other financial institutions, but the information is only available for clients who have a loan in our sample. The number of rows in the table for each loan in the sample matches the number of credits the client had in the Credit Bureau before the application date. The table contains a total of 1,716,428 data points.

- *bureau_balance.csv*

  The presented table exhibits the monthly balances of prior credits that were reported to the Credit Bureau, and it comprises 27,299,925 entries. The data frame contains no missing values. Each previous credit has a row for every recorded month in history. To be precise, the number of rows in the table is calculated by multiplying the number of loans in the sample by the related previous credits' number and the number of months with available historical data for each previous credit.

4.1.2 Descriptive Statistics

In reference to the missing values of the given data, the following table displays information about the number of entries, columns, and the columns with the highest amount of missing values, along with their corresponding proportions.

Data frame	# entries	# columns	Highest missing value attribute	Highest proportion missing value
application_test	48,744	121	COMMONAREA_MEDI, COMMONAREA_AVG, COMMONAREA_MODE	68.7161%
application_train	307,511	122	COMMONAREA_MEDI, COMMONAREA_AVG, COMMONAREA_MODE	69.8723%
previous_application	1,670,214	37	RATE_INTEREST_PRIVILEGED, RATE_INTEREST_PRIMARY	99.64370%
bureau	1,716,428	17	AMT_ANNUAITY	71.4735%
credit_card_balance	3,840,312	23	AMT_PAYMENT_CURRENT	19.9981%
POS_CASH_balance	10,001,358	8	CNT_INSTALMENT_FUTURE	0.2608%
installments_payments	13,605,401	8	DAYS_ENTRY_PAYMENT, AMT_PAYMENT	0.0214%
bureau_balance	27,299,925	3	None	0%

Table 4.1: The proportion of missing values on each variable in training and testing datasets.

The train set contains 282,686 non-default loans and 24,825 default loans, resulting in a default rate of approximately 8.07%.

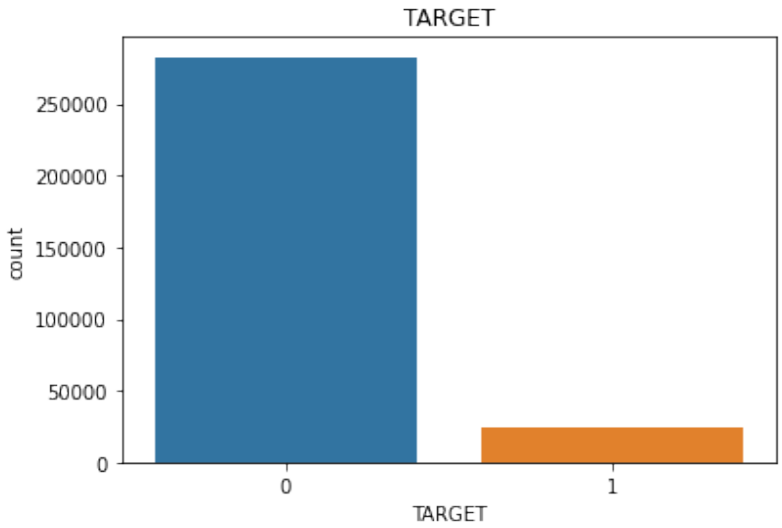


Figure 4.2: The distribution of defaults and non-defaults in the training set.

Regarding the contract type of the applications, there are two kinds of loans consisting of revolving and cash loans. Revolving loan is a credit type that permits borrowers to access a prearranged amount of credit and pay it back over time, often with interest, an example of revolving loans is credit cards (Hamilton and Khan 2001). On the other hand, cash loan is a type of loan where the borrower receives a lump sum of money from the lender, which is then repaid in installments over a fixed period of time. The default rate of cash loan and revolving loan are 8.35% and 5.48% respectively according to the given data. The distribution of default and non-default by contract type is as follows.

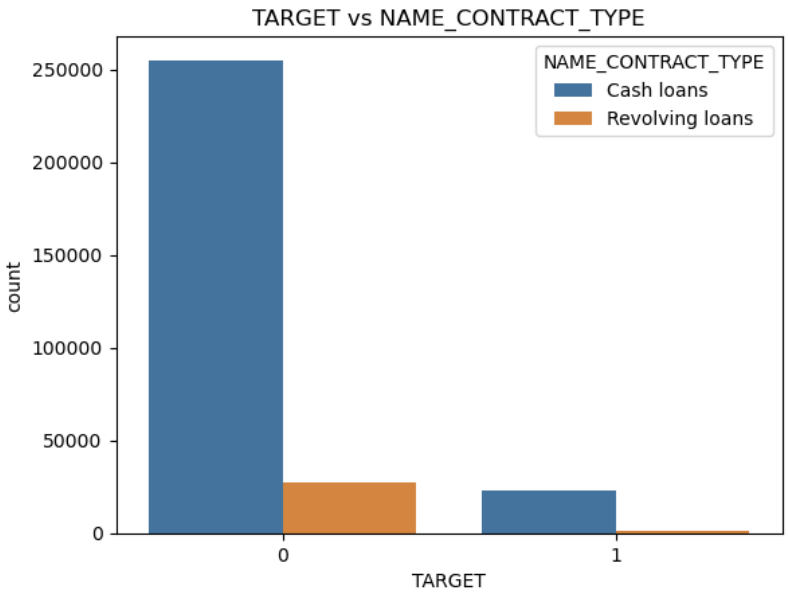


Figure 4.3: The distribution of defaults and non-defaults in the training set with respect to contract types.

With respect to applicant's gender, numerous studies, such as those conducted by Anthony and Horne (2003) and D. Chen, Liao, and Yizao Liu (2017), have demonstrated that females have a lower default rate compared to males. The distribution of default and non-default based on gender can be visualized using the following chart, with M denoting male, F denoting female, and XNA representing missing values. The default rate of female in the data file is 7% which is much less than 10.14% of male.

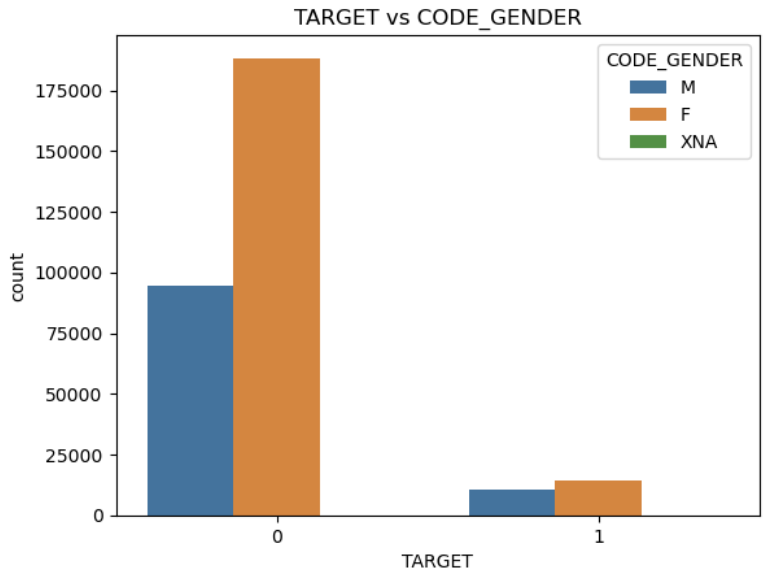


Figure 4.4: The distribution of defaults and non-defaults in the training set is broken down with respect to genders.

About the relationship between family status and solvency, the following diagram shows that married individuals in the dataset have greater responsibilities compared to those in other family statuses.

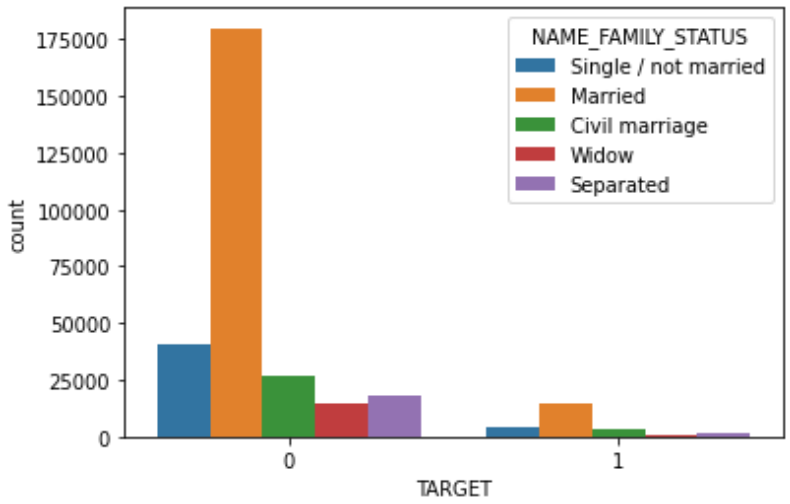


Figure 4.5: The distribution of defaults and non-defaults in training set with respect to family status.

Relating to the loan application amount, the following histogram illustrates how it relates to customers' ability to repay. The histogram suggests that there is little correlation between the loan application amount and repayment ability.

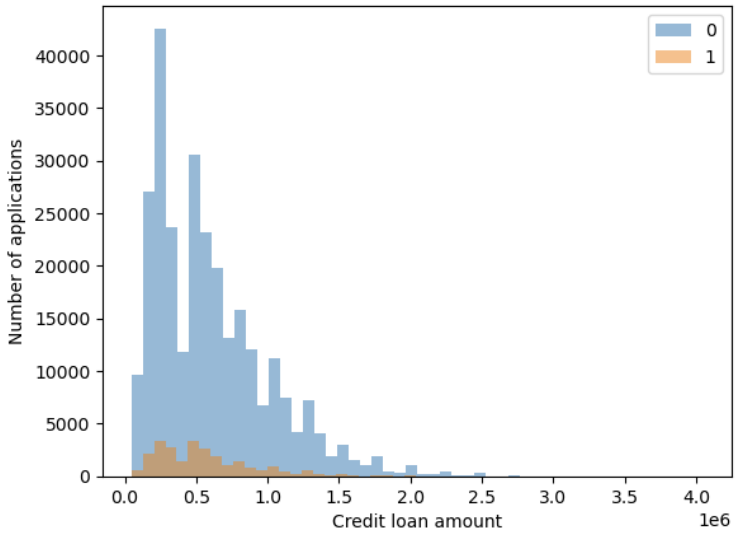


Figure 4.6: The histogram of the distribution of loan amount based on the target variable.

4.2 Pre-processing data

The process of data preparation is critical as there may be outliers and errors in the data that could significantly impact the model's results. The approach taken to handle the data will have a significant influence on the model's outcome.

4.2.1 Feature engineering

In the context of credit risk modeling, feature engineering is critical for identifying the most informative and predictive features and creating a set of features that best captures the underlying patterns and relationships in the data. The Home Credit Default Risk dataset poses several challenges due to its large size, significant amount of missing data, and the inclusion of mixed data types such as both text and numerical values. Given the crucial impact of payment history on future loan repayment performance, numerous ratio-based features have been computed which can provide a better indication of the borrower's financial situation. Additionally, feature selection, scaling, and normalization can help improve the performance of machine learning algorithms on this dataset. However, in order to apply boosting methods, all data types must be numerical. As a result, all text data has been

converted to numerical data using methods such as ordinal, one-hot, and dummy variable encoding. This conversion allows these data to be applicable for other algorithms as well.

The performance of machine learning models such as Random Forest, Decision Tree, and Logistic Regression on the dataset was suboptimal, with the maximum achievable AUC being only 50%. However, after applying feature engineering techniques, the maximum AUC value increased significantly to 74%, highlighting the crucial role of feature engineering in improving the performance of predictive models on the dataset.

In addition, all data files except for *application* have been grouped based on the key SK_ID_CURR to generate the final grouped data. In the resulting *application_train* and *application_test* data sets, there are 443 and 442 columns, respectively.

4.2.2 Slicing data technique

The thesis utilizes complex and resource-intensive algorithms that necessitate data slicing. To achieve this, a slicing technique called stratified sampling is employed, which reduces the training data to 40% of its original size. Stratified sampling is a statistical sampling technique that involves dividing a population into subgroups (strata) based on a characteristic, and drawing samples from each stratum in proportion to its size. This technique is commonly used when subgroups in the population have different characteristics, and there is a need to ensure that the resulting sample is representative of the entire population.

To implement stratified sampling for all variables in the dataset, the study divides the data into subgroups based on the values of each variable and samples from each stratum in proportion to its size. This ensures that the resulting sample is representative of the entire population for all variables in the dataset.

4.2.3 Resampling Methods

To address class imbalance problems, it is crucial to rebalance the class distribution using resampling methods, which traditional machine learning algorithms

may not consider, leading to inadequate performance on the minority class (Xiao et al. 2021). This thesis employed three resampling techniques, namely the Random Under-Sampling, Random Over-Sampling, and Synthetic Minority Over-sampling Technique, to rebalance the dataset.

Specifically, Random Under-Sampling (RUS) is a method that seeks to balance the class distribution by randomly removing samples from the majority class. Although this technique can effectively reduce the dominance of the majority class, it may also result in the loss of informative data. In contrast, Random Over-Sampling (ROS) is a technique that balances the class distribution by randomly copying samples from the minority class. While this approach can be effective in enhancing the presence of the minority class, it has the potential to cause overfitting and diminish the ability to generalize. SMOTE is an oversampling method, similar to ROS, that produces synthetic samples from the minority class by interpolating between adjacent samples. To create synthetic samples, SMOTE identifies the minority class' k-nearest neighbors and interpolates between them to form a new sample. This technique is useful in increasing the minority class representation while maintaining the original dataset distribution.

4.3 Experimental Setup

To perform the analysis, I developed custom code in Python using the scikit-learn library. The code used to implement the ensemble methods is available at *GitHub repository*, Thu (2023), and includes functions for data preprocessing, model training, and evaluation. A stratified 5-fold cross-validation methodology was employed to assess the performance of the ensemble methods, and a comparative analysis was conducted against traditional machine learning models.

4.3.1 Cross-validation

In order to address the issue of class imbalance in imbalanced datasets, it is important to employ cross-validation techniques. This study utilized both stratified k-fold and k-fold cross-validation methods to tackle the problem. Both techniques

involve dividing the dataset into k equal-sized folds, which are then utilized to train and evaluate the model k times. However, the key difference between the two methods lies in the way the data is partitioned (Pedregosa et al. 2018).

Specifically, a prevalent method for performing k-fold cross-validation is to randomly partition the dataset into k folds of equal size, without taking into account the class distribution. Nonetheless, this can result in certain folds having a significantly different class distribution than the overall dataset.

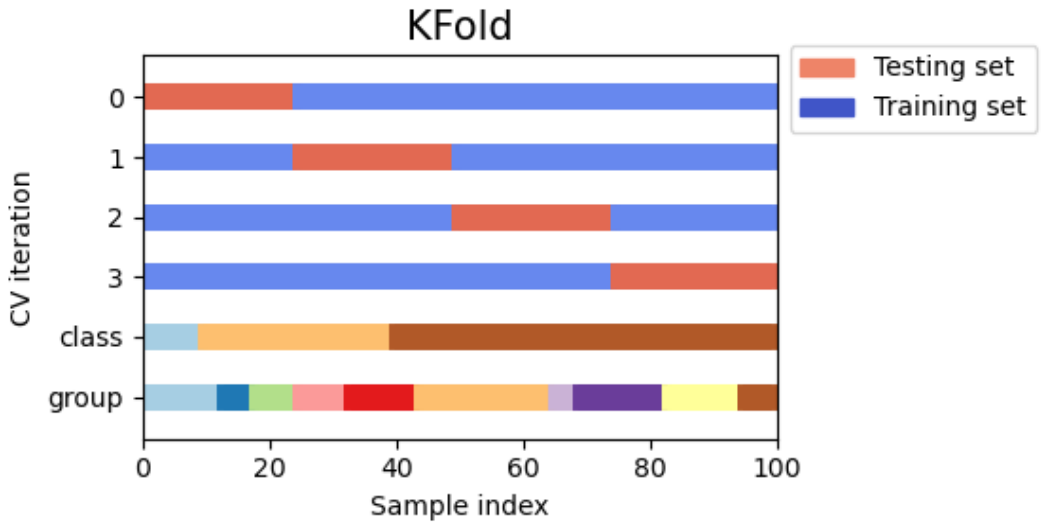


Figure 4.7: K Fold Visualization Cross-Validation.

On the other hand, stratified k-fold cross-validation aims to create folds that are representative of the class distribution in the entire dataset. To accomplish this, the data is initially divided into strata according to the target class, and subsequently, each fold is formed by randomly selecting samples from each stratum. This technique is useful in guaranteeing that each fold has a comparable class distribution to the overall dataset.

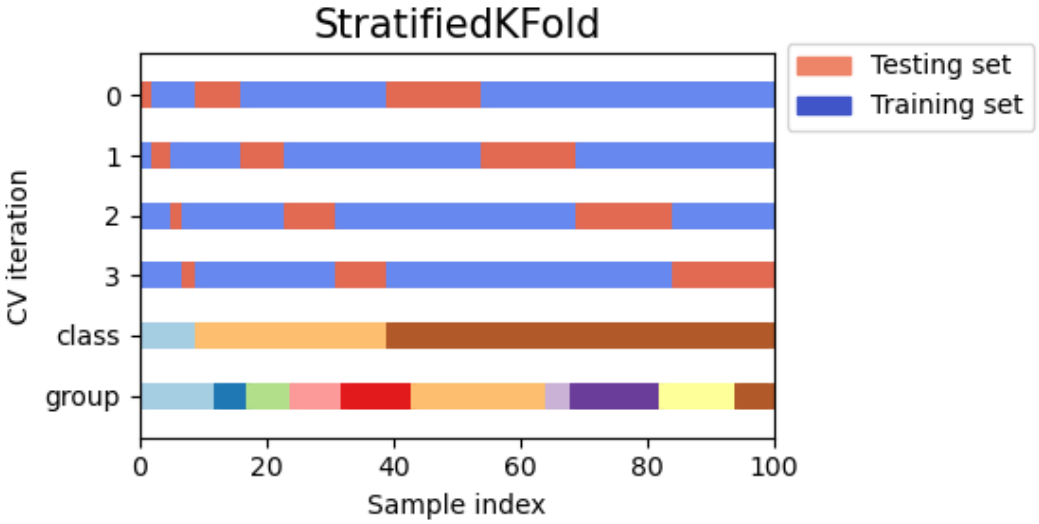


Figure 4.8: Stratified K Fold Visualization Cross-Validation.

4.3.2 Baseline models

Motivation

In this section, the performance of the proposed model was compared against several standard classification algorithms, referred to as baseline models. The aim of this comparison was to provide an understanding of the proposed model's performance and demonstrate its efficacy relative to simpler models. The baseline models included logistic regression, k-NN, SVM, decision tree, and neural network models, which are commonly utilized in classification problems. Through this comparison, valuable insights into the effectiveness of the proposed model and its potential for practical applications can be gained.

Rationale

The reasons for selecting these models as baseline models are as follows:

- Prevalence: These models are extensively utilized in machine learning for classification, making them one of the most prevalent algorithms in the field. Consequently, they are frequently utilized as a standard reference point for comparison in research studies.

- Simplicity: The algorithms are relatively simple and easy to interpret (logistic regression, k-NN, decision tree).

- Nonlinearity and Flexibility: Neural network and SVM are capable of modeling nonlinear relationships between the input features and output labels. This can be especially important in cases where the relationship between the features and labels is complex or difficult to model. Furthermore, both offer a great deal of flexibility in terms of the model architecture and hyperparameters.

- Interpretability: Apart from their simplicity, certain models such as decision tree has a high level of interpretability. This attribute can be advantageous in comprehending the model's predictive logic and recognizing significant data characteristics.

- Diversity: A varied range of methods for classification are embodied in the models, spanning from linear models such as logistic regression to non-parametric models such as k-NN. Through a comparative analysis of the models, the advantages and limitations of different classification approaches can be comprehended.

- Benchmarking: A benchmark for performance can be established by comparing the performance of the proposed model with well-established and widely used models. This will provide context for the practical significance of the results obtained.

Description

- Logistic Regression: To account for the presence of highly correlated attributes in logistic regression, Principal Component Analysis (Jolliffe 2002) was applied to reduce the dimensionality of the data. Through dimensionality reduction, the data has been compressed to 170 dimensions while retaining 99.82% of the original variance.

- k-NN: The method was used to classify the data point, which involved using the Euclidean distance metric and the five nearest neighbors. The algorithm

determined the five closest neighbors to the query point and made a prediction by taking a majority vote. It is important to consider that increasing the value of nearest neighbors can create a more resilient model that is less affected by noise in the data, but it might also result in overfitting. The decision to select the Euclidean distance metric was based on its simplicity and time efficiency when compared to other metrics.

- SVM: The kernel function that is used is Radial Basis Function since it is able to model complex decision boundaries that are not linearly separable in the input space.

- Decision tree: The maximum depth of the decision tree is set to 20, and a fixed seed value of 20 is used to initialize the internal random number generator. This guarantees that the model will generate the same sequence of random numbers each time it is executed with a random state value of 20.

- Neural network: The model defines a neural network architecture with five hidden layers. These hidden layers have 128, 256, 512, 1024, and 2048 nodes, respectively. Each hidden layer is followed by a dropout layer with a dropout rate of 0.2. The dropout layers are designed to prevent overfitting by randomly dropping out 20% of the nodes in each layer during training. The final layer is an output layer with a single node and a softmax activation function, which is used for binary classification.

4.3.3 Ensemble models

Bagging models

- Random Forest: This has been configured with 1000 decision trees, each with a maximum depth of 10 levels, and using the Gini impurity measure to determine the quality of splits. As the number of trees increases, so does the amount of time required for computation. Furthermore, to prevent overfitting, the model requires a minimum of 10 samples for splitting internal nodes and a minimum of 30 samples for being at a leaf node.

- Bagged Neural Network: This ensemble classifier uses a neural network architecture called multilayer perceptron (MLP) with four hidden layers of different sizes (256, 128, 64, and 32). The logistic function is used as the activation function, and the adaptive moment estimation (Adam) algorithm is used as the solver. The logistic function maps input values to outputs between 0 and 1, making it ideal for binary classifications where the output must be either 0 or 1. The Adam algorithm is a popular optimization algorithm that uses gradient descent with momentum and adaptive learning rates to handle complex models and large datasets. This model also includes early stopping to prevent overfitting, a maximum of 200 iterations, and a learning rate of 0.008. The bagging classifier includes 100 estimators, bootstrap sampling, and a random state of 42. Progress updates during training are displayed with a verbose parameter set to 500. As a result of using the logistic function as the activation function and the Adam algorithm as the solver, this model can effectively classify binary data with high accuracy.

Boosting models

- AdaBoost: Specific parameters were used to implement the model, including a decision tree as the weak classifier limited to a maximum depth of 5. The number of weak classifiers to be trained was set to 200. The learning rate parameter controls the effect of each weak classifier on the final model and was set to 0.08.

- Gradient Boosting: The ensemble contained 100 decision trees. The learning rate, which determines the step size during gradient descent and can affect the convergence speed, was set to 0.008. The maximum depth of each decision tree, which can impact the model's ability to generalize to new data, was limited to 5. To ensure reproducibility of results, the random state parameter was set to 0. The loss parameter was also specified as "deviance", indicating the loss function used for optimization.

- XGBoost: During training, the model utilized 5 threads. The ensemble con-

sisted of a maximum of 50,000 decision trees. The step size taken during gradient descent was set to 0.008. The maximum depth of each decision tree was restricted to 15, while the number of leaves in each decision tree was set to 100. Additionally, only 50% of the features were used in each tree.

- Light GBM: The model utilized 5 threads for parallel processing. It performed a total of 50,000 boosting rounds, with each round contributing 0.008 to the final model. The model's trees had a maximum depth of 12 and 58 leaves. Each tree used 0.613 of the columns, and each boosting round used 0.8 of the samples. The maximum number of bins for each feature was set to 410. The L1 and L2 regularization terms were set to 3.564 and 4.930, respectively. The minimum sum of instance weights and the minimum number of samples required to form a leaf node were 6 and 165, respectively. Finally, the silent and verbose parameters were set to -1 to suppress output and control the level of verbosity of the model output.

Stacking models

Top 5 models with the highest AUC were selected as meta learners, which include Extreme Gradient Boosting (XGBoosting), Light Gradient Boosting Machine (Light GBM), Random Forest, Gradient Boosting, and Logistic Regression. The meta-learners utilize the top 8 results with the highest AUC on the test set as input. Both k-fold cross-validation and stratified k-fold cross-validation of Light GBM, and XGBoosting with preprocessed data that underwent ROS, RUS, and SMOTE treatments.

4.4 Results

As some models took a significant amount of time to train, a reduced 40% of the train set was used. The sample data was obtained using stratified shuffle split method, which maintains the class distribution in both the training and testing sets. The data is first split into training and testing sets based on a prescribed ratio, and then randomly shuffled to ensure that the samples in both sets are representative

of the overall population. This technique is particularly useful when dealing with imbalanced datasets where the classes are not uniformly distributed. By using this splitting method, the resulting models can be trained on a representative subset of the data, enabling them to make accurate predictions on new data. The following table displays the AUC score of each model on the test set, obtained after training on only 40% of the available training data.

		Cross-validation								Average AUC	Max time duration
		Stratified K Fold (5 folds)				K Fold (5 folds)					
		SMOTE	ROS	RUS	NONE	SMOTE	ROS	RUS	NONE		
Baseline	PCA - Logistic Regression	70.43%	71.26%	70.83%	71.13%	70.65%	71.43%	71.51%	71.29%	71.16%	6h
	kNN	54.44%	53.52%	55.19%	53.49%	53.6%	53.13%	55.31%	53.02%	53.89%	3h
	SVM			60.14%	50.27%			60.82%	**50.00%**	55.31%	11h
	Decision tree	64.47%	64.99%	66.49%	68.18%	62.16%	63.24%	58.29%	67.95%	64.47%	5m
	NN	**50.00%**	**50.00%**	**50.00%**	**50.00%**	**50.00%**	**50.00%**	**50.00%**	**50.00%**	**50.00%**	10h
Bagging	Random Forest	70.65%	74.81%	74.49%	74.79%	70.69%	74.82%	74.57%	74.8%	74.14%	10h
	Bagged Neural Network			56.28%	55.4%			56.05%	55.63%	55.84%	11h
Boosting	AdaBoost	51.11%	64.47%	65.26%	50.96%	51.03%	65.28%	66.04%	50.98%	59.15%	11h
	GBoost	69.99%	72.16%	72.07%	71.68%	69.96%	72.03%	71.96%	71.65%	71.64%	10h
	XGBoost	77.01%	77.1%	76.44%	77.2%	76.99%	77.28%	76.68%	77.27%	76.99%	10h
	Light GBM	67.18%	76.98%	76.15%	77.60%	66.17%	76.63%	76.42%	77.41%	75.34%	5m
Stacking	Logistic Regression	77.22%	77.24%	77.25%	77.26%	77.23%	77.26%	77.23%	77.28%	77.25%	5m
	Random Forest	77.37%	77.62%	77.64%	77.64%	77.32%	77.62%	77.62%	77.67%	77.59%	15m
	GBoost	77.53%	77.57%	77.47%	77.68%	77.5%	77.67%	77.63%	**77.77%**	77.61%	15m
	XGBoost	77.55%	77.49%	77.65%	77.67%	77.65%	77.53%	77.68%	77.71%	77.62%	15m
	Light GBM	77.64%	77.63%	77.56%	77.55%	77.53%	77.62%	77.6%	77.69%	77.60%	1m
Average AUC		68.59%	71.4%	69.93%	68.03%	68.46%	70.82%	69.09%	68.64%		

Table 4.2: The performance of ensemble methods on testing dataset with feature engineering.

Table 4.2 indicates that stacking models outperform baseline, bagging, and boosting models. Among the stacking models, the meta learner - Gradient Boosting - achieves the highest AUC score of 77.77%. However, Random Forest, GBoost, XGBoost, and Light GBM have similar AUC scores, possibly due to the limited input values that range from 76.44% to 77.6%.

When it comes to baseline models, logistic regression with dimensionality reduction data shows the best performance with an average AUC score of 71.16%, whereas Neural Network perform the worst with an average AUC score of 50%. The training of SVM is extremely time-consuming due to its complex computational algorithm, particularly with a large number of variables and data points. As a result, there are no results for SMOTE and ROS methods.

Boosting methods were assessed, and Light GBM with 5-fold cross-validation

achieved the highest AUC score of 77.60%. However, XGBoost performs better in terms of average AUC score, as Light GBM is not effective on randomly under-sampled data. Without resampling and cross-validation techniques, AdaBoost performs poorly with an average AUC score of about 59%. Additionally, when using SMOTE resampling methods, AdaBoost, GBoost, and Light GBM exhibit poorer performance compared to other resampling methods, while XGBoost performs worse with RUS resampling methods.

In regard to bagging methods, Random Forest outperforms Bagged Neural Network with an average AUC score of 74.14%. However, training a bagged neural network is also highly time-consuming, particularly with a large number of variables and data points, and there are no results for SMOTE and ROS methods.

The results also demonstrate that ROS outperforms other resampling methods. Regarding cross-validation techniques, there is not much difference between stratified k-fold and traditional k-fold.

It is clear that complex algorithms such as SVM, Neural Network, Random Forest, Bagged Neural Network, AdaBoost, GBoost, and XGboost consume a significant amount of time to train large data sets. The Light GBM model is one of the most impressive models as it produces high AUC scores within a short amount of time.

4.5 Limitations

Several limitations were identified during the course of this study, which are outlined as follows.

First, there is the limited generalizability of the findings to other credit scoring contexts, as the study focused on a specific credit application dataset from a single financial institution. This could limit the external validity of the results, despite the large sample size. It is uncertain whether the findings will apply to different populations or settings.

Second, there exists the possible influence of missing data on the results. Imputation techniques were employed to handle missing values, but these techniques are not flawless and could have introduced bias into the analysis. Furthermore, the

utilization of imputation may have restricted the interpretability of the results, as imputed data may not precisely reflect the actual values.

Third, the study depended on a restricted range of features to forecast creditworthiness, which could have restricted the ability to capture significant predictors of credit risk. Although feature selection techniques were utilized to determine the most significant features, it is conceivable that other features, not considered in the analysis, might be relevant for credit scoring.

Fourth, the study solely concentrated on ensemble methods for credit scoring and did not compare the results with other machine learning or statistical techniques, except for the algorithms mentioned in this thesis. Although the findings indicated that the stacking models outperformed other methods, there is a possibility that other methods may perform better for particular credit scoring contexts.

Fifth, the utilization of intricate machine learning models like Gradient Boosting and Random Forest could have restricted the interpretability of the results. Although these models can attain high accuracy, they can be challenging to understand and may not offer clear insights into the factors influencing credit scores. Subsequent studies may consider balancing the trade-off between AUC and interpretability when choosing models for credit scoring.

Sixth, there are the limited resources available for highly time-consuming algorithms. Due to the computational demands of some of the algorithms used in the analysis, such as ensemble methods, it may not be feasible to run them on machines with limited computational power or time. This could potentially limit the scale or scope of the study and the ability to investigate larger datasets or more complex models.

Seventh, there would be the potential impact of tuning parameters on the results. While tuning parameters can improve the performance of machine learning models, it can be time-consuming and may require significant expertise. Therefore, it may not be possible to exhaustively explore all possible parameter values, leading to the possibility of suboptimal performance. Additionally, different parameter settings may be optimal for different credit scoring contexts, which could limit the generalizability of the results.

Eighth, a potential statistical limitation of the study is the use of only AUC as the performance metric. While AUC is a widely used metric for evaluating binary classification models, it may not capture all aspects of model performance, such as the balance between sensitivity and specificity or the ability to correctly classify different levels of credit risk. Therefore, the reliance on AUC alone may not provide a comprehensive assessment of the performance of the models, and other metrics may need to be considered, such as precision, recall, or F1-score.

Last but not least, machine learning models employed for credit scoring may pose potential concerns regarding fairness, bias, and discrimination. For instance, these models may unintentionally discriminate against specific demographic groups or reinforce existing biases present in the data. Additionally, the utilization of sensitive personal information for credit scoring may elicit apprehensions about privacy and data protection. Thus, it is crucial to consider the ethical implications of using machine learning models for credit scoring and to promote fairness and transparency in the development and application of these models.

Chapter 5

Conclusion and Discussion

The purpose of this study was to examine the effectiveness of ensemble methods in credit scoring models as compared to traditional algorithms. The results showed a significant difference between the performance of ensemble algorithms, particularly between stacking algorithms and the traditional ones. Interestingly, the majority of the models performed well even without resampling methods, as measured by the average AUC. Notably, the traditional model, Logistic Regression, had an impressive performance that exceeded that of several complex models, such as Neural Network, Bagged Neural Network, and AdaBoost. It is worth mentioning that the computation time was lengthy, given the large size of the dataset in both dimensions. However, Light GBM was able to produce a high AUC score in a relatively short amount of time, which is a considerable advantage when dealing with big data.

Although the study showed promising results, it is important to acknowledge its limitations. Firstly, the dataset used was restricted to a specific financial institution and period, and therefore, the findings may not be applicable to other contexts or populations. Secondly, the study's use of a limited range of features and complex machine learning models may have restricted the ability to capture significant predictors of credit risk and interpret the results. Thirdly, the study's reliance on AUC as the sole performance metric may not provide a comprehensive assessment of the models' performance. Fourthly, the study only tested a limited number of ensemble methods, and future research could explore additional methods to improve credit scoring accuracy further. Finally, it is also necessary to consider the ethical

implications of using machine learning models for credit scoring, as they may lead to concerns regarding fairness, bias, and discrimination. Therefore, it is essential to promote fairness and transparency in the development and application of these models.

According to the study, employing ensemble methods in credit scoring models can notably improve precision and reduce computational time. This discovery holds great significance for financial institutions and other entities that use credit scoring models to make crucial decisions. Further studies can expand on these findings to enhance credit scoring accuracy and establish industry standards for organizations in this domain.

Bibliography

Abellán, Joaquín and Carlos J. Mantas (2014). "Improving experimental studies about ensembles of classifiers for bankruptcy prediction and credit scoring". In: *Expert Systems with Applications* 41.8, pp. 3825–3830. ISSN: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2013.12.003. URL: https://www.sciencedirect.com/science/article/pii/S0957417413009676.

Anthony, Denise and Christine Horne (2003). "Gender and Cooperation: Explaining Loan Repayment in Micro-Credit Groups". In: *Social Psychology Quarterly* 66.3, pp. 293–302. ISSN: 01902725. URL: http://www.jstor.org/stable/1519827 (visited on 03/15/2023).

Baesens, B. et al. (2003). "Benchmarking State-of-the-Art Classification Algorithms for Credit Scoring". In: *The Journal of the Operational Research Society* 54.6, pp. 627–635. ISSN: 01605682, 14769360. URL: http://www.jstor.org/stable/4101754 (visited on 03/12/2023).

Breiman, Leo (1996). "Bagging predictors". In: *Machine learning* 24, pp. 123–140.

Chen, Dongyu, Hua Liao, and Yizao Liu (2017). "Gender discrimination in online peer-to-peer credit lending: evidence from a lending platform in China". In: *Electronic Commerce Research* 17.4, pp. 553–583.

Chen, Tianqi and Carlos Guestrin (2016). "Xgboost: A scalable tree boosting system". In: *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pp. 785–794.

Dumitrescu, Elena Ivona et al. (2022). "Machine Learning for Credit Scoring: Improving Logistic Regression with Non Linear Decision Tree Effects". In: *European Journal of Operational Research* 297.3, pp. 1178–1192. DOI: 10.1016/j.ejor.2021.06.053. URL: https://hal.science/hal-03331114.

Fitzpatrick, Alex (July 2015). *The History of Credit Scores*. English. URL: https://shorturl.at/gqFPQ.

Freund, Yoav, Robert Schapire, and Naoki Abe (1999). "A short introduction to boosting". In: *Journal-Japanese Society For Artificial Intelligence* 14.771-780, p. 1612.

Freund, Yoav, Robert E Schapire, et al. (1996). "Experiments with a new boosting algorithm". In: *icml*. Vol. 96. Citeseer, pp. 148–156.

Friedman, Jerome H. (2001). "Greedy Function Approximation: A Gradient Boosting Machine". In: *The Annals of Statistics* 29.5, pp. 1189–1232. ISSN: 00905364. URL: http://www.jstor.org/stable/2699986 (visited on 03/18/2023).

GeeksforGeeks (2021). *AUC-ROC Curve*. https://www.geeksforgeeks.org/auc-roc-curve/. [Online; accessed 21-June-2023].

Hambali, Moshood et al. (2019). "ADABOOST ensemble algorithms for breast cancer classification". In: *Journal of Advances in Computer Research* 10.2, pp. 31–52.

Hamilton, R. and M. Khan (2001). "Revolving Credit Card Holders: Who Are They and How Can They Be Identified?" In: *The Service Industries Journal* 21.3, pp. 37–48. DOI: 10.1080/714005031. URL: https://doi.org/10.1080/714005031.

Hastie, Trevor et al. (2009). *The elements of statistical learning: data mining, inference, and prediction*. Vol. 2. Springer.

He, Haibo and Edwardo A. Garcia (2009). "Learning from Imbalanced Data". In: *IEEE Transactions on Knowledge and Data Engineering* 21.9, pp. 1263–1284. DOI: 10.1109/TKDE.2008.239.

HomeCredit (2018). *Home Credit Default Risk*. https://www.kaggle.com/c/home-credit-default-risk.

Hsieh, Nan-Chen and Lun-Ping Hung (2010). "A data driven ensemble classifier for credit scoring analysis". In: *Expert Systems with Applications* 37.1, pp. 534–545. ISSN: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2009.05.059. URL: https://www.sciencedirect.com/science/article/pii/S0957417409004771.

James, Gareth et al. (2013). *An introduction to statistical learning*. Vol. 112. Springer.

Jolliffe, Ian T (2002). *Principal component analysis for special types of data*. Springer.

Ke, Guolin et al. (2017). "LightGBM: A Highly Efficient Gradient Boosting Decision Tree". In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Long Beach, California, USA: Curran Associates Inc., pp. 3149–3157. ISBN: 9781510860964.

Khandani, Amir E., Adlar J. Kim, and Andrew W. Lo (2010). "Consumer credit-risk models via machine-learning algorithms". In: *Journal of Banking and Finance* 34.11, pp. 2767–2787. ISSN: 0378-4266. DOI: https://doi.org/10.1016/j.jbankfin.2010.06.001. URL: https://www.sciencedirect.com/science/article/pii/S0378426610002372.

Kiviat, Barbara (2019). "Credit scoring in the United States". In: *economic sociology_the european electronic newsletter* 21.1, pp. 33–42.

Liu, Wanan, Hong Fan, and Meng Xia (2022). "Tree-based heterogeneous cascade ensemble model for credit scoring". In: *International Journal of Forecasting*. ISSN: 0169-2070. DOI: https://doi.org/10.1016/j.ijforecast.2022.07.007. URL: https://www.sciencedirect.com/science/article/pii/S0169207022001054.

Liu, Yan et al. (2009). "A comparative study of credit scoring methods based on support vector machines". In: *Expert Systems with Applications* 36.2, pp. 3302–3308.

Opitz, David and Richard Maclin (Dec. 1999). "Popular Ensemble Methods: An Empirical Study". In: 11. DOI: 10.1613/jair.614.

Pedregosa, F. et al. (2018). *Plotting Cross-Validated Estimator Scores*. https://scikit-learn.org/stable/auto_examples/model_selection/plot_cv_indices.html. [Online; accessed 21-June-2023].

Quinlan, J. Ross (1986). "Induction of decision trees". In: *Machine learning* 1, pp. 81–106.

Santos, Miriam Seoane et al. (2018). "Cross-Validation for Imbalanced Datasets: Avoiding Overoptimistic and Overfitting Approaches [Research Frontier]". In:

IEEE Computational Intelligence Magazine 13.4, pp. 59–76. DOI: 10.1109/MCI.2018.2866730.

Schapire, Robert E. (2013). "Explaining AdaBoost". In: *Empirical Inference*. Springer Berlin Heidelberg, pp. 37–52. DOI: 10.1007/978-3-642-41136-6_5. URL: https://doi.org/10.1007%2F978-3-642-41136-6_5.

Steele, Brian, John Chandler, and Swarna Reddy (2016). *Algorithms for data science*. Springer.

Suthaharan, S. (2015). *Machine Learning Models and Algorithms for Big Data Classification: Thinking with Examples for Effective Learning*. Integrated Series in Information Systems. Springer US. ISBN: 9781489976413. URL: https://books.google.com.vn/books?id=ad3HCgAAQBAJ.

Thu, Nguyen Thi Minh (2023). *GitHub repository*. https://github.com/Chorlie/ensemble-method-for-credit-scoring-model-on-home-credit-default-risk. [Software].

Wang, Yadong et al. (2022). "Deep reinforcement learning with the confusion-matrix-based dynamic reward function for customer credit scoring". In: *Expert Systems with Applications* 200, p. 117013.

Wang, Yuelin et al. (2020). "A Comparative Assessment of Credit Risk Model Based on Machine Learning — a case study of bank loan data". In: *Procedia Computer Science* 174, pp. 141–149.

Wolpert, David H (1992). "Stacked generalization". In: *Neural networks* 5.2, pp. 241–259.

Xiao, Jin et al. (2021). "Impact of resampling methods and classification models on the imbalanced credit scoring problems". In: *Information Sciences* 569, pp. 508–526. ISSN: 0020-0255. DOI: https://doi.org/10.1016/j.ins.2021.05.029. URL: https://www.sciencedirect.com/science/article/pii/S0020025521004874.

Zhou, Zhi-Hua (2012). *Ensemble methods: Foundations and algorithms*. CRC press.